(총괄·1 세부) 지능형 디지털 트윈 연합 운용 및 예측 핵심기술 개발

연합트윈 기술해설서

온톨로지 기반 디지털 트윈 검색

티오이십일콤즈

2026-08-25

문서 정보

구분	내용
문서명	연합트윈 기술해설서 온톨로지 기반 디지털 트윈 검색
발행일	2026-08-25
작성 주체	티오이십일콤즈
대상 저장소	https://github.com/etri/FeDiT.git
대상 모듈	OPFW/ontology_digital_twin_search

작성 기준

본 문서는 GitHub 저장소 `https://github.com/etri/FeDiT.git` 를 기준으로 기술합니다.

프론트엔드 소스 및 운용 자원이 GitHub 저장소에 반영 완료된 상태를 전제로 합니다.

본문의 `{ }` 표기는 실제 운용 환경 값으로 바꾸어 입력하는 부분입니다.

목차

문서 정보 2
작성 기준 2
목차 3
1. 개요 7
가. 문서의 목적 및 활용 방법 7
○ 작업 상황별 수행 범위 7
나. 시스템 구성 7
○ 구동되는 컨테이너 9 개 8
다. 배포 프로세스 8
2. 개발 환경 구성 10
가. 개발 환경 정보 10
○ 백엔드 10
○ 프론트엔드 10
나. 사전 준비 사항 11
○ 작업 PC 최소 사양 11
○ 작업 PC 에 설치가 필요한 도구 12
○ 설치 후 버전 확인 12
○ Windows 환경 사전 설정 (필수) 12
○ 사전에 확보해야 하는 정보 13
○ 로컬 인프라 접속 정보 13
○ 로컬 인프라 문제 해결 14
다. 문서에서 사용하는 용어 14
라. 표기 규칙 15
○ 명령 프롬프트 여는 방법 15
3. 소스 코드 내려받기 15
가. 내려받을 위치 이동 15
나. Git 저장소 Clone 15
○ 필요한 모듈만 내려받기 16
다. 내려받은 폴더 구조 16
○ 대상 모듈의 주요 폴더 구성 17

○ 운용 자원 구성 (`initResource/OntologyConverter/`) 17
4. 백엔드 19
가. 이 장에서 수행하는 작업 19
○ 인프라 구동 19
나. 프로젝트 열기 20
다. 빌드 전 설정 확인 23
라. 빌 드 25
○ 명령행에서 빌드하는 경우 25
○ IntelliJ 에서 빌드하는 경우 25
마. 빌드 산출물 확인 및 반영 26
5. 프론트엔드 27
가. 프로젝트 경로 이동 27
나. 패키지 설치 28
다. 로컬 구동 확인 28
라. 초기 데이터 적재 (최초 설치 시) 28
○ 실행 여부 판단 28
○ 초기 적재·온톨로지 생성 API 29
○ 제공되는 화면 30
마. 서버 URL 설정 (빌드 전 필수) 30
바. 빌 드 31
사. 빌드 산출물 확인 및 반영 31
6. 서버 배포 33
가. 이 장에서 수행하는 작업 33
나. 서버 접속 및 사전 준비 33
○ 1) 서버 접속 33
○ 2) 시스템 사전 점검 34
○ 3) 시각 동기화 34
○ 4) Docker 설치 35
○ 5) docker 그룹 등록 35
○ 6) SELinux 설정 36
○ 7) 포트 선점 확인 36
○ 8) 설치 검증 36

다. 운용 폴더 이관 37
○ SELinux 컨텍스트 적용 (Rocky / RHEL 계열 필수) 39
○ 요청 전달(프록시) 설정 39
라. 설정 파일 수정 40
○ 포트를 변경해야 하는 경우 41
마. 컨테이너 기동 41
바. 구동 상태 확인 42
○ 로그 확인 42
○ 서버 내부 응답 확인 42
○ Neo4j 데이터 확인 42
○ 웹 브라우저 확인 주소 43
○ 배포 확인 체크리스트 43
사. 컨테이너 중지 및 재기동 44
○ 새 버전 배포 시 44
아. 문제 해결 44
7. 부록 46
가. 신규 서버 구축 순서 요약 46
나. 백엔드 갱신 배포 순서 요약 46
다. 화면 갱신 배포 순서 요약 46
라. 개발·배포 도구 설치 상세 46
1) Git 47
2) JDK 17 (Eclipse Temurin) 49
3) Node.js 50
4) Docker Desktop (+ WSL2) 51
5) IntelliJ IDEA 54
6) Visual Studio Code 55
7) DBeaver 56
8) 7-Zip 57
9) SSH · SFTP 접속 도구 57
마. [대안] Docker 미사용 시 인프라 직접 설치 58
바. 자주 사용하는 명령어 59
사. 다운로드 링크 총괄 60

아. 설정 항목 일람 61
자. 인터넷이 차단된 환경에서의 설치 (폐쇄망) 61
○ 1) Docker 설치 패키지 반입 61
○ 2) 컨테이너 이미지 반입 62
차. 프록시 환경에서의 Docker 설정 63
카. 사내망(프록시) 환경에서의 작업 PC 설정 63

1. 개요

가. 문서의 목적 및 활용 방법

본 문서는 연합트윈 온톨로지 기반 디지털 트윈 검색 시스템을 소스 코드 내려받기 단계부터 운용 서버 배포 및 구동 확인 단계까지 수행하기 위한 절차를 기술한 기술해설서입니다. 백엔드(온톨로지 변환기)와 프론트엔드(검색·그래프 화면)를 모두 포함하며, 실제 입력해야 하는 명령어와 정상 여부 확인 방법을 순서대로 기술하였으므로 앞에서부터 차례대로 따라 수행하시면 됩니다.

별도의 사전 지식이 없더라도 수행할 수 있도록 각 단계마다 수행 목적, 입력 명령어, 정상 여부 확인 방법을 함께 기술하였습니다. 본문에 등장하는 용어는 2 장 「다. 문서에서 사용하는 용어」에, 명령어 표기 방식은 2 장 「라. 표기 규칙」에 정리되어 있습니다.

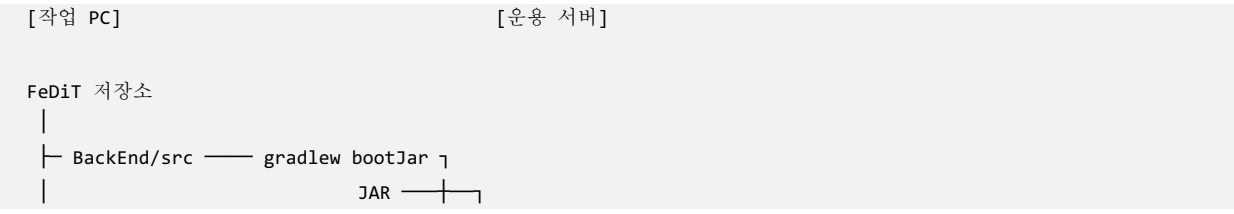
○ 작업 상황별 수행 범위

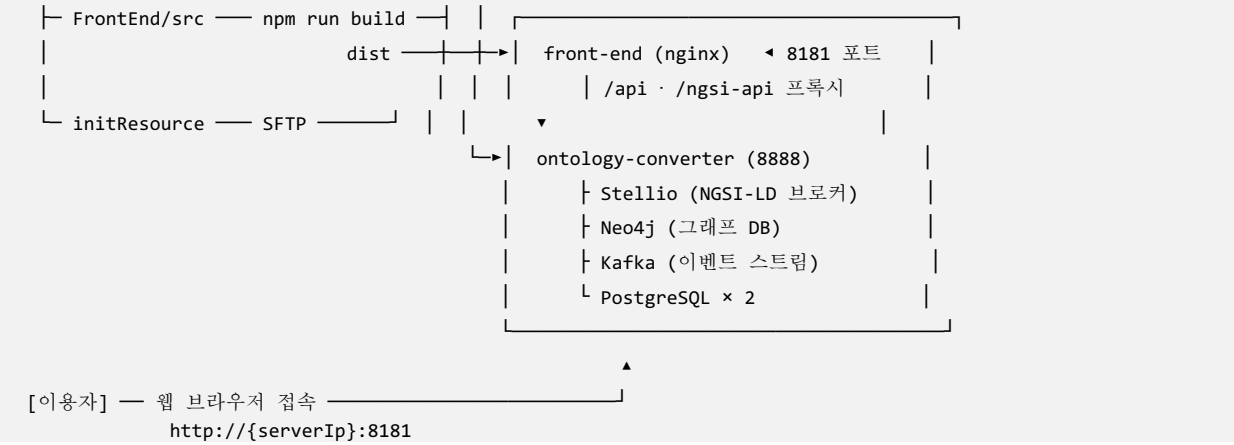
작업 상황	설명	수행 범위
신규 서버 구축	서버에 아무것도 구성되어 있지 않은 상태에서 처음 배포하는 경우	2 장 ~ 6 장 (전체 수행)
배포만 수행	소스를 수정하지 않고 저장소에 포함된 산출물로 배포하는 경우	3 장, 6 장 (4·5 장 생략)
백엔드 갱신 배포	백엔드 소스를 수정하여 교체하는 경우	3 장, 4 장, 6 장 「다」~「마」
화면 갱신 배포	프론트엔드 화면만 최신 버전으로 교체하는 경우	3 장, 5 장, 6 장 「다」~「마」
서비스 재기동	서버의 서비스를 중지 / 재기동해야 하는 경우	6 장 「사」

나. 시스템 구성

온톨로지 기반 디지털 트윈 검색 시스템은 **백엔드(온톨로지 변환기)**, **프론트엔드(React SPA)**, 그리고 **인프라 구성요소 7 종**으로 이루어진 총 **9 개 컨테이너**로 구동됩니다.

백엔드는 외부 원본(fedit API)에서 받은 디지털 트윈 메타 정보를 NGSI-LD 엔티티로 변환해 Stellio 브로커에 적재하고, 이를 온톨로지(TTL)로 생성하여 Neo4j 그래프로 동기화합니다. 프론트엔드는 그 결과를 그래프 시각화 화면과 자연어 검색 화면으로 제공합니다.





이용자의 브라우저는 서버의 8181 포트로 접속합니다. 화면 파일뿐만 아니라 조회 요청도 이 포트를 통해 프록시로 전달되므로, 이용자 PC 에서 서버의 8181 포트로 접근할 수 있어야 합니다.

> ※ 그래프 시각화 화면은 브라우저가 **Neo4j(7687 Bolt)에 직접 접속**하여 그립니다. 다른 PC 에서 그래프를 보려면 7687 포트도 함께 개방되어야 합니다.

○ 구동되는 컨테이너 9 개

구분	컨테이너 명	이미지	호스트 포트	역할
프론트	front-end	nginx:alpine (자체 빌드)	8181:80	화면 전달 및 API 프록시
백엔드	ontology-converter	자체 빌드 (temurin:17-jre)	8888:8888	온톨로지 변환·검색 API
그래프 DB	ontology-neo4j	neo4j:5.22.0	7474 / 7687	온톨로지 그래프 저장
NGSI-LD	stellio-api-gateway	stellio/stellio-api-gateway	8085:8080	Stellio 진입점
NGSI-LD	stellio-search-service	stellio/stellio-search-service	8083	엔티티 조회
NGSI-LD	stellio-subscription-service	stellio/stellio-subscription-service	8084	구독 관리
메시지	stellio-kafka	apache/kafka:3.8.0	29092	이벤트 스트림
DB	stellio-postgres	stellio/stellio-timescale-postgis	5432	Stellio 엔티티·시계열
DB	ontology-converter-postgres	postgres:17	6543:5432	메시지 staging

> ※ **호스트 포트와 컨테이너 포트가 다른 항목에 주의합니다.** Kafka 는 컨테이너 안에서 `kafka:9092` 로 통신하지만 호스트에서는 **29092** 로, Stellio API Gateway 는 컨테이너 안에서 `api-gateway:8080` 이지만 호스트에서는 **8085** 로 접속합니다. 호스트에서 개발할 때 이 값을 혼동하면 연결에 실패합니다.
>
> ※ **설정 파일에 적는 주소는 위 표의 「컨테이너 명」이 아니라 docker-compose.yml 의 서비스 명입니다.** 아래 5 개는 두 이름이 서로 다르므로 주의합니다. 4 장 「다」의 `bolt://neo4j:7687`, `kafka:9092`, `http://api-gateway:8080` 이 모두 서비스 명 기준입니다.

다. 배포 프로세스

컨테이너 명 (docker compose ps 표시)	도커 네트워크 주소 (서비스 명)

컨테이너 명 (docker compose ps 표시)	도커 네트워크 주소 (서비스 명)
stellio-kafka	kafka
stellio-api-gateway	api-gateway
stellio-search-service	search-service
stellio-subscription-service	subscription-service
ontology-neo4j	neo4j

배포는 아래 6 단계로 진행됩니다. 백엔드·프론트엔드가 접속하는 외부 주소는 **빌드 시점에 산출물 안에 포함**되므로, 빌드 이전에 반드시 운용 환경에 맞게 주소를 변경해야 합니다.

단계	작업 구분	주요 내용	사용 도구 / 명령어	해당 장
1 단계	사전 준비	필요 도구 설치 및 접속 정보 확보	-	2 장
2 단계	소스 코드 내려받기	GitHub 저장소로부터 프로젝트 소스 Clone	`git clone`	3 장
3 단계	백엔드 빌드	설정 수정 후 JAR 생성	`gradlew bootJar`	4 장
4 단계	프론트엔드 빌드	설정 수정 후 배포용 정적 파일 생성	`npm run build`	5 장
5 단계	운용 폴더 이관	산출물 배치 후 운용 폴더를 서버로 전송	`scp` / SFTP(WinSCP)	6 장「다」
6 단계	서버 기동·검증	컨테이너 기동 및 구동 확인	`docker compose`	6 장「마」~「바」

※ 소스를 수정하지 않고 **배포만 수행**하는 경우에는 3·4 단계를 생략할 수 있습니다. 저장소의 `initResource/OntologyConverter/` 에 빌드된 JAR 과 dist 가 이미 포함되어 있습니다.

2. 개발 환경 구성

가. 개발 환경 정보

○ 백엔드

구분	내용
언어	Java 17
프레임워크	Spring Boot (※ 아래 주의사항 참고)
빌드 도구	Gradle (프로젝트에 `gradlew` 래퍼 포함)
데이터베이스	PostgreSQL 17 (메시지 staging), Neo4j 5.22.0 (그래프)
메시지 큐	Apache Kafka 3.8.0 (KRaft)
연계 브로커	Stellio Context Broker (NGSI-LD)
주요 라이브러리	Apache Jena 5.0(온톨로지 생성), Spring Data Neo4j, Spring Data JPA, WebSocket
개발 도구	IntelliJ IDEA

> ※ **빌드 재현성 주의** — `build.gradle` 은 현재 Spring Boot 정식 릴리스가 아닌 **`4.0.0-SNAPSHOT`** 을 사용하며, 의존성을 `https://repo.spring.io/snapshot` 에서 내려받습니다.
>
> ```gradle
> id 'org.springframework.boot' version '4.0.0-SNAPSHOT'
> repositories { mavenCentral(); maven { url = 'https://repo.spring.io/snapshot' } }
> ```
>
> 스냅샷은 예고 없이 교체·삭제되므로 **같은 소스를 다시 빌드해도 결과가 달라지거나 실패할 수 있으며**, 폐쇄망에서는 빌드가 불가능합니다. 운용 배포를 목적으로 한다면 **정식 릴리스 버전으로 고정할 것을 권장**합니다. 부득이하게 유지해야 한다면 최초 빌드 성공 시점의 의존 라이브러리를 사내 저장소(Nexus 등)에 캐시해 두어야 합니다.
>
> ※ 소스 수정 없이 배포만 수행하는 경우에는 이 제약이 적용되지 않습니다.

○ 프론트엔드

구분	내용
언어	TypeScript
런타임	Node.js 20.19 이상 (22 LTS 권장)
프레임워크	React
빌드 도구	Vite 7
그래프 시각화	Neo4j 직접 접속 (Bolt)
운용 웹 서버	Nginx (정적 파일 서빙 및 API 프록시)
개발 도구	Visual Studio Code

> ※ Vite 7 은 Node.js 20.19 미만 버전을 지원하지 않습니다.

> ※ 프론트엔드 소스 폴더에 포함된 `Dockerfile` 에는 `FROM node:18` 이 지정되어 있으나, 실제 배포는 호스트에서 빌드한 `dist` 를 nginx 이미지에 복사하는 방식을 사용하므로 프론트엔드 폴더 내의 Dockerfile 은 사용하지 않습니다.

나. 사전 준비 사항

○ 작업 PC 최소 사양

본 문서의 4 장 「가」에서 작업 PC 에 인프라 컨테이너를 기동하고, 백엔드·프론트엔드 빌드를 함께 수행합니다. 아래 기준을 밑도는 PC 에서는 빌드가 매우 느려지거나 컨테이너 기동에 실패할 수 있습니다.

> ※ 소스를 수정하지 않고 배포만 수행하는 경우에는 로컬 인프라를 기동하지 않으므로 위 기준보다 낮은 사양에서도 가능합니다.

항목	기준	근거
운영체제	Windows 10 (64bit) 이상	본 문서의 명령어는 Windows 기준입니다
CPU	4 코어 이상	로컬 인프라 컨테이너와 빌드를 동시에 수행
메모리	16GB 권장 (최소 8GB)	Docker Desktop 에 8GB 를 할당하고 그 위에서 빌드 수행
여유 디스크	20GB 이상	저장소 · 컨테이너 이미지 9 종 · node_modules 누적

○ 작업 PC 에 설치가 필요한 도구

구분	도구	버전	용도
필수	Git	2.30 이상	소스 내려받기
필수	JDK (Eclipse Temurin)	17	백엔드 빌드
필수	Node.js	20.19 이상 (22 LTS 권장)	프론트엔드 빌드
필수	Docker Desktop	Engine 24 이상	이미지 빌드·실행
필수	SSH 접속 도구	-	서버에 명령 입력 (6 장 「나」) — PuTTY
필수	SFTP 파일 전송 도구	-	운용 폴더 업로드 (6 장 「다」) — WinSCP
권장	압축 도구 (7-Zip)	최신	대용량 zip 해제 (Windows 기본 도구는 경로 길이 오류 발생)
선택	IntelliJ IDEA (Community)	2024.1 이상	백엔드 소스 확인·수정
선택	Visual Studio Code	최신	프론트엔드 소스 확인·수정
선택	DBeaver	최신	PostgreSQL 데이터 확인

> ※ 각 도구의 상세 설치 절차(다운로드 화면, 설치 옵션, 명령어(winget) 설치, 환경변수 설정 등)는 「부록 라. 개발·배포 도구 설치 상세」에 수록하였습니다. 처음 구성하는 PC 라면 부록 「라」의 winget 사용 준비 → 1) Git → 8) 7-Zip 순서로 설치·설정을 마친 뒤 이 절로 돌아와 아래 버전 확인부터 이어서 수행하십시오.

○ 설치 후 버전 확인

필수 도구들이 전부 설치되었다면, 각 버전을 확인할 수 있도록 아래 내용을 한 줄씩 입력한다. 입력 시 버전이 확인되지 않고, 명령, 프로그램 또는 배치 파일이 아니라는 메시지가 나오면 cmd 창을 닫고 다시 시도해보거나 PC 재부팅을 시도한다.

```
git --version        # 예: git version 2.45.2.windows.1
java -version        # openjdk version "17.x.x"
node --version       # v22.x.x (또는 v20.19 이상)
npm --version
docker --version
docker compose version
```

○ Windows 환경 사전 설정 (필수)

본 저장소에는 경로가 긴 파일이 포함되어 있어 Windows 의 기본 경로 길이 제한(260 자)에 걸려 Clone 이 실패할 수 있습니다. **Clone 이전에** 아래 설정을 먼저 수행하십시오.

```
git config --global core.longpaths true
```

> ※ Docker Desktop 은 7 장 부록 「라」 4)의 .wslconfig 명령으로 WSL2 메모리를 8GB 이상 할당합니다. 컨테이너 9 개 중 JVM 기반 서비스가 많아 메모리가 부족하면 기동 중 일부 컨테이너가 재시작될 수

> 있습니다.

로컬에서 인프라 컨테이너를 기동하기 전에, 사용할 포트를 다른 프로그램이 이미 쓰고 있는지 확인합니다.

```
netstat -ano | findstr /C:":5173 " /C:":5432 " /C:":6543 " /C:":7474 " /C:":7687 " /C:":8083 " /C:":8084 " /C:":8085 " /C:":8888 " /C:":29092 "
```

> ※ 아무 줄도 출력되지 않으면 정상입니다. 위 10 개는 로컬 인프라 컨테이너(8 개)와 백엔드(8888) · 프론트 개발 서버(5173)가 사용하는 포트입니다. 출력되는 줄이 있으면 맨 오른쪽 숫자가 해당 프로그램의 PID 이므로 tasklist | findstr {PID} 로 확인한 뒤, 그 프로그램을 종료하거나 docker-compose.yml 의 호스트 포트를 변경하십시오.

○ 사전에 확보해야 하는 정보

구분	필요 정보	용도
Git	저장소 주소 (공개 저장소는 계정 불필요)	3 장 소스 코드 내려받기
서버	서버 IP, SSH 계정 / 비밀번호, SSH 포트	6 장 서버 접속
서버	SFTP 접속 정보 (SSH 계정 · 포트와 동일)	6 장 파일 업로드
연계 서버	**fedit API 서버 주소 / 포트**	4 장 설정 수정, 6 장 초기 데이터 적재

> ※ fedit API 주소는 초기 데이터 적재의 원본입니다. **서버에서 접근 가능한 주소**여야 하며, 접근이 불가능하면 데이터가 적재되지 않아 화면이 비어 있게 됩니다.

○ 로컬 인프라 접속 정보

4 장「가」에서 기동하는 로컬 인프라 컨테이너 7 개의 접속 정보입니다. 아래 값은 BackEnd₩src₩searchRule₩.env 에 정의되어 있으며, DB 확인 도구나 브라우저로 직접 접속할 때 사용합니다.

> ※ 위 계정은 저장소에 포함된 초기값입니다. 운용 서버에 배포할 때는 6 장「라」에서 교체하십시오.

구성 요소	로컬 접속 주소	계정
Neo4j Browser	http://localhost:7474	neo4j / test1234
Neo4j Bolt	bolt://localhost:7687	neo4j / test1234
Stellio API 게이트웨이	http://localhost:8085/ngsi-ld/v1	인증 없음
Stellio PostgreSQL	localhost:5432 (DB stellio_search)	stellio / stellio_password
변환기 PostgreSQL	localhost:6543 (DB ontology_converter)	ontology_converter / ontology1234
Kafka	localhost:29092	인증 없음
백엔드 API (로컬 실행 시)	http://localhost:8888	인증 없음
프론트 개발 서버	http://localhost:5173	인증 없음

> ※ Stellio search(8083) · subscription(8084) 은 내부 통신용이라 직접 접속할 일이 없습니다. 기동 여부만 docker compose ps 로 확인합니다.

○ 로컬 인프라 문제 해결

> ※ 서버 배포 단계의 문제 해결은 6 장「아」를 참고하십시오.

증상	원인	조치
Cannot connect to the Docker daemon	Docker Desktop 이 실행되지 않음	Docker Desktop 실행 후 docker info 가 정상 출력되는지 확인
port is already allocated	해당 포트를 다른 프로그램이 사용 중	위「Windows 환경 사전 설정」의 포트 확인 명령으로 점유 프로그램 확인
컨테이너가 계속 재시작됨	Docker 에 할당된 메모리 부족	Docker Desktop 메모리를 8GB 이상으로 조정 (부록「라」4)
stellio search · subscription 이 Exited	DB 준비 전에 먼저 기동되어 종료됨	DB 가 healthy 가 된 뒤 docker compose up -d 로 다시 기동

다. 문서에서 사용하는 용어

용어	설명
빌드(build)	소스 코드를 실행 가능한 파일로 변환하는 작업. 백엔드는 `gradlew bootJar`, 프론트엔드는 `npm run build` 로 수행합니다.
JAR	백엔드 빌드 시 생성되는 Java 실행 파일. Docker 이미지에 포함됩니다.
dist	프론트엔드 빌드 시 생성되는 폴더. 서버에 올리는 실제 배포 대상입니다.
Docker	프로그램을 실행 환경까지 통째로 묶어 구동하는 도구.
이미지(image)	프로그램과 실행 환경을 묶어 놓은 설치 파일에 해당합니다.
컨테이너(container)	이미지를 실행한 상태. 실제로 동작 중인 프로그램에 해당합니다.
Docker Compose	여러 개의 컨테이너를 설정 파일(.yml) 하나로 함께 기동하는 도구.
볼륨(volume)	컨테이너를 지워도 데이터가 남도록 Docker 가 관리하는 저장 공간.
프록시(proxy)	이용자의 요청을 대신 받아 실제 처리 서버로 전달하는 역할. 본 문서에서는 Nginx 가 수행합니다.
NGSI-LD	사물·상황 정보를 표현하는 표준 데이터 모델. Stellio 가 이 표준을 구현한 브로커입니다.
Stellio	NGSI-LD 표준을 구현한 오픈소스 컨텍스트 브로커. 엔티티 저장·조회를 담당합니다.
온톨로지(TTL)	개념과 관계를 기술한 지식 표현. 본 시스템은 NGSI-LD 엔티티를 TTL 로 변환해 Neo4j 에 적재합니다.
Neo4j	노드와 관계를 저장하는 그래프 데이터베이스. 그래프 화면의 데이터 원천입니다.
Bolt	Neo4j 전용 접속 프로토콜. 브라우저가 7687 포트로 직접 접속합니다.
SSH	서버에 원격으로 접속하여 명령어를 입력하기 위한 접속 방식.
FTP	작업 PC 의 파일을 서버로 전송하기 위한 파일 전송 방식.
운용 폴더	서버에서 서비스를 구동하는 기준 폴더. 저장소의 `initResource/OntologyConverter/` 를 서버로 옮겨 `/usr/local/OntologyConverter` 로 둔 것입니다. 6 장의 명령어는 모두 이 폴더에서 실행합니다.

라. 표기 규칙

표기	의미	예시
{ }	실제 환경에 맞는 값으로 바꾸어 입력하는 부분 (중괄호는 입력하지 않음)	`{serverIp}` → 운용 서버 주소
#	명령어 뒤에 붙는 설명(주석). 입력하지 않아도 됩니다.	`java -version  # 버전 확인`
\	명령어가 다음 줄로 이어짐을 의미합니다. 한 줄로 이어서 입력해도 됩니다.	
회색 배경 영역	터미널에 그대로 입력하는 명령어 또는 파일 내용	
※ 표기	반드시 확인해야 하는 주의 사항	

구분	입력 위치	본 문서의 해당 장
작업 PC	IntelliJ / VS Code 의 터미널 또는 PC 의 터미널	3 장, 4 장, 5 장
운용 서버	SSH 로 서버에 접속한 상태의 터미널	6 장

○ 명령 프롬프트 여는 방법

본 문서의 명령어는 별도 표기가 없으면 명령 프롬프트(CMD) 기준입니다.

일반 권한 — 탐색기에서 해당 폴더로 이동한 뒤 주소창에 cmd 를 입력하고 Enter 를 누릅니다. 그 폴더에서 명령창이 열립니다.

관리자 권한 — 검색창에 cmd 를 입력한 뒤 Ctrl + Shift + Enter 를 누릅니다. (또는 우클릭 → [관리자 권한으로 실행])

도구 설치(winget) 와 WSL2 준비는 관리자 권한이 필요합니다. 그 밖의 빌드·실행 명령은 일반 권한으로 충분합니다.

3. 소스 코드 내려받기

가. 내려받을 위치 이동

```
cd /d C:\
```

> ※ Windows 의 경로 길이 제한(260 자)을 피하기 위해 드라이브 최상위(C:₩)에서 바로 내려받습니다. (2 장「나」참고)

나. Git 저장소 Clone

```
git clone https://github.com/etri/FeDiT.git
cd /d C:\FeDiT
```

`etri/FeDiT` 가 공개 저장소로 전환된 이후에는 별도의 계정 정보나 토큰(PAT) 입력 없이

내려받을 수 있습니다.

※ 공개 전환 이전에는 접근 권한이 필요합니다. 저장소 관리자에게 GitHub 계정을 협업자(Collaborator)로 등록 요청한 뒤, PAT(repo 권한) 또는 SSH 키로 인증하십시오. 등록되지 않은 계정은 저장소가 조회되지 않으며 "repository not found" 오류가 표시됩니다.

이미 내려받은 저장소가 있는 경우에는 Clone 대신 아래 명령어로 최신 소스를 반영합니다.

```
cd /d C:\FeDiT
git pull origin main
```



[그림 1] GitHub 저장소 Clone

○ 필요한 모듈만 내려받기

```
cd /d C:\
git clone --filter=blob:none --no-checkout --branch main https://github.com/etri/FeDiT.git
cd /d C:\FeDiT
git sparse-checkout set --cone OPFW/ontology_digital_twin_search
git checkout
```

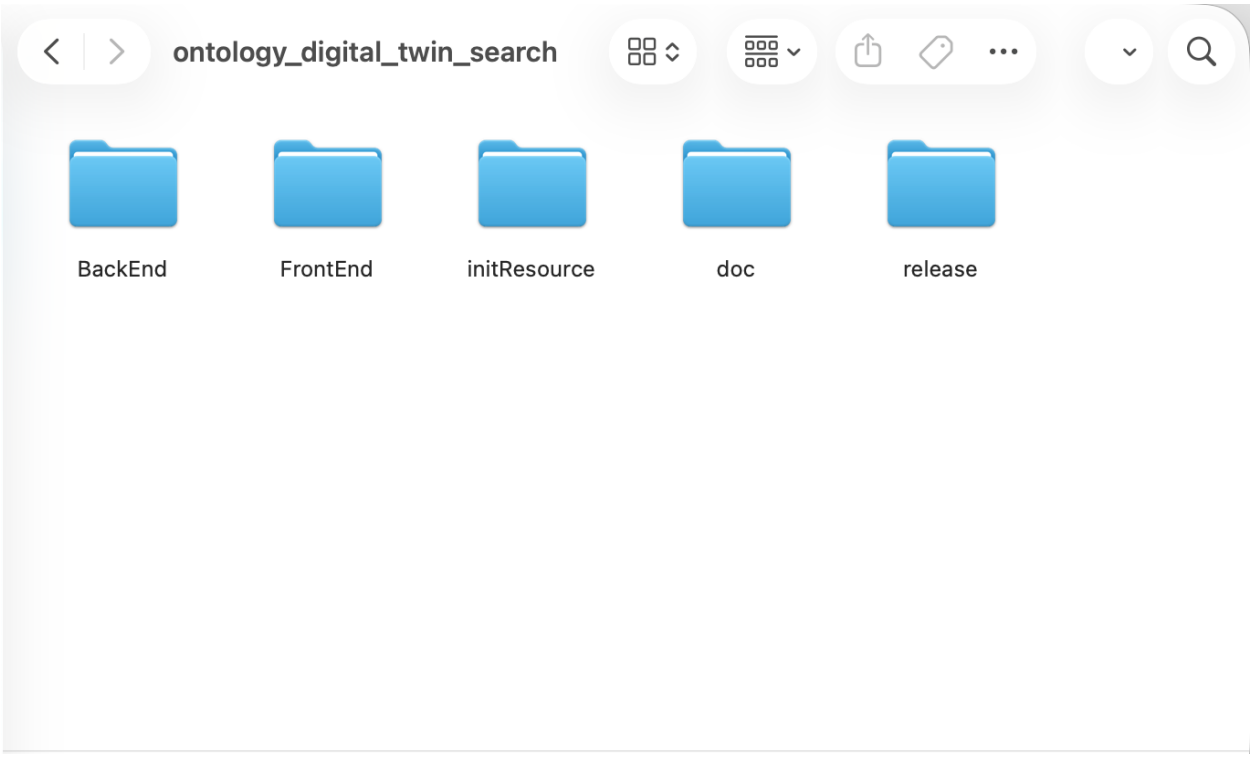
※ 저장소 전체를 내려받으면 용량이 크므로, 본 문서의 대상 모듈만 내려받는 위 방식을 권장합니다.

다. 내려받은 폴더 구조

Clone 이 완료되면 C:₩FeDiT 아래에 다음 구조가 생성됩니다. 본 문서의 대상은 OPFW₩ontology_digital_twin_search 입니다.

```
C:\FeDiT\
├── CDFW/                                  연합 데이터 프레임워크
├── Doc/                                   전체 문서
├── OSFW/                                  개방형 서비스 프레임워크
├── OPFW/                                  연합 운용 프레임워크
│   ├── msa_device_api_gateway/
│   ├── msa_device_orchestrator/
│   ├── msa_dynamic_event_tracker/
│   └── ontology_digital_twin_search/      ◂ 본 문서의 대상 모듈
│       ├── doc/                           기술 문서
│       ├── initResource/
│       │   └── OntologyConverter/         ◂ 운용 폴더 (6 장에서 서버로 이관)
```

```
│       ├── release/                          배포 산출물 보관
│       ├── BackEnd/
│       │   └── src/
│       │       └── searchRule/                    ◂ 백엔드 소스 (4 장)
│       └── FrontEnd/
│           └── src/
│               └── ontology-converter-frontend/    ◂ 프론트엔드 소스 (5 장)
├── AUTHORS.md
├── LICENSE
└── README.md
```



[그림 2] Clone 완료 후 폴더 구조

○ 대상 모듈의 주요 폴더 구성

폴더 / 파일	설명
doc/	기술해설서 및 설계서
initResource/OntologyConverter/	도커 배포에 필요한 파일만 모인 폴더. 이 구성 그대로 배포할 수 있습니다
release/	배포 산출물 보관 폴더
BackEnd/src/searchRule/	백엔드 프로젝트 루트 (build.gradle 위치)
FrontEnd/src/ontology-converter-frontend/	프론트엔드 프로젝트 루트 (package.json 위치)

○ 운용 자원 구성 (`initResource/OntologyConverter/`)

```
initResource/
```

```
└ OntologyConverter/              ← 운용 폴더 (서버 이관 대상)
    ├ docker-compose.yml         컨테이너 9 개 정의
    ├ .env                       계정·DB 명·이미지 태그
    ├ Dockerfile                 백엔드 이미지 정의
    ├ searchRule-0.0.1-SNAPSHOT.jar   ★ 백엔드 빌드 산출물
    ├ output/                    ★ TTL 산출물 저장 폴더 (반드시 존재해야 함)
    └ frontend/
        ├ Dockerfile             프론트 이미지 정의
        ├ nginx.conf             정적 서빙 + API 프록시
        └ dist/                  ★ 프론트 빌드 산출물
```

★ 표시는 4·5 장에서 생성해 배치하는 항목이고, 나머지는 저장소에 이미 포함된 설정 파일입니다.

> ※ 저장소에는 **빌드된 JAR 과 dist 가 이미 포함**되어 있습니다. 소스를 수정하지 않고 배포만 수행하는 경우 4·5 장을 생략하고 6 장으로 바로 진행할 수 있습니다.
>
> ※ **`output/` 폴더는 반드시 존재해야 합니다.** `docker-compose.yml` 이 `./output:/app/data/output` 으로 마운트하며, 폴더가 없으면 Docker 가 root 소유로 자동 생성하여 백엔드가 TTL 파일을 쓰지 못할 수 있습니다.
>
> ※ output/ 은 .gitignore 에 등록되어 있어 Git 저장소와 배포용 zip 에 포함되지 않습니다. Clone 직후 이 폴더가 보이지 않는 것이 정상이며, 서버 이관 후 6 장 「다」의 sudo mkdir -p /usr/local/OntologyConverter/output 으로 생성합니다.

4. 백엔드

가. 이 장에서 수행하는 작업

백엔드 소스를 빌드하여 **실행 파일(JAR)** 을 생성합니다. 본 장을 완료하면 운용 폴더에 최신 JAR 이 준비된 상태가 됩니다.

순서	작업	해당 절
1	개발 도구에서 프로젝트를 엽니다.	나
2	빌드 전 설정확인을 합니다.	다
3	빌드를 수행합니다.	라
4	산출물을 확인하고 운용 폴더에 반영합니다.	마

> ※ 소스 수정 없이 저장소에 포함된 JAR 을 그대로 사용하는 경우 본 장을 생략할 수 있습니다.

○ 인프라 구동

백엔드는 Neo4j·Kafka·Stellio·PostgreSQL 에 접속합니다. 따라서 **백엔드를 IntelliJ 에서 실행하기전에, 인프라를 먼저 docker 로 띄워줘야 합니다.**

```
docker info
```

먼저 Docker Desktop 이 실행 중인지 확인합니다. 오류 없이 정보가 출력되면 준비된 상태입니다.

백엔드 폴더에서 아래 명령으로 인프라 컨테이너들을 기동합니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search\BackEnd\src\searchRule
docker compose up -d
```

위 docker compose up -d 실행 후, 컨테이너 7 개가 모두 정상 기동되었는지 확인한 다음 백엔드를 실행합니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search\BackEnd\src\searchRule
docker compose ps                          # State 가 running/healthy 인지 확인
docker compose logs --tail 50 neo4j        # 특정 서비스 로그 확인
docker compose logs -f                     # 실시간 전체 로그 (종료: Ctrl + C)
```

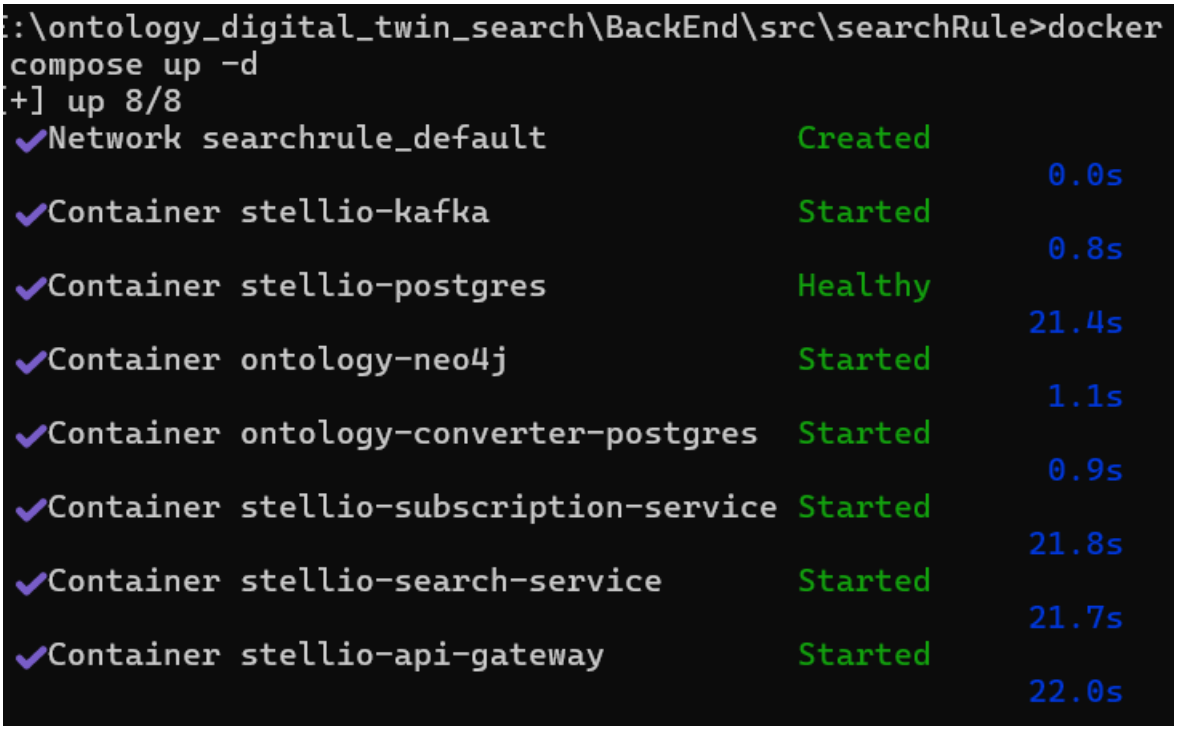
작업을 마친 뒤 인프라를 내릴 때는 아래를 입력합니다.

(-v 는 데이터 볼륨까지 삭제하므로 초기화가 필요할 때만 사용)

```
docker compose stop                        # 중지 (데이터 유지)
docker compose down                        # 중지 + 컨테이너 삭제
# docker compose down -v                  # 볼륨까지 삭제 (데이터 전부 초기화)
```

> ※ 이 compose 파일 하단에는 백엔드(ontology-converter) 서비스가 주석 처리되어 함께 들어 있습니다. 백엔드까지 컨테이너로 기동하려면 해당 부분의 주석을 해제한 뒤 docker compose up -d --build 를 실행합니다. 접속 주소·포트는 application-docker.properties 가 자동으로 전환하므로 별도 수정이 필요하지 않으며, 기동 전에 4 장 「라」에서 생성한 JAR(build₩libs₩searchRule-0.0.1-SNAPSHOT.jar)과 output 폴더가 있는지만 확인하면 됩니다. output 폴더는 도커가 자동생성하지만, 안전하게 진행하려면

> 경로 내 mkdir output 명령으로 만듭니다.



나. 프로젝트 열기

IntelliJ 로 열지 않고 명령창에서 바로 실행하려면, 프로젝트 폴더(build.gradle 위치)로 이동한 뒤 Gradle 래퍼로 기동합니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search\BackEnd\src\searchRule
java -version                                # 17 인지 확인
gradlew.bat bootRun                          # 종료는 실행중인 창에서 Ctrl + c
```

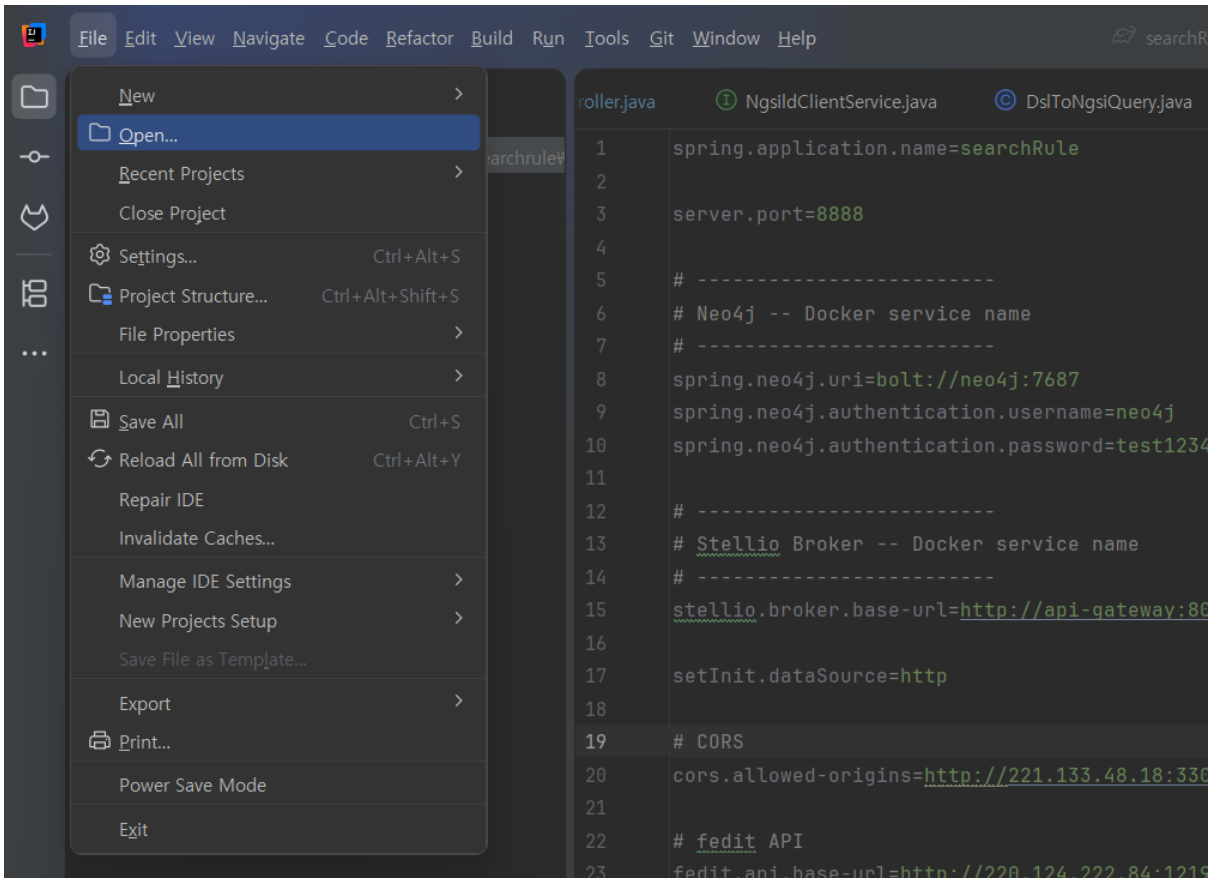
○ IntelliJ 에서 여는 경우

IntelliJ IDEA 의 [Open] 에서 아래 폴더(build.gradle 이 있는 폴더)를 선택해 엽니다. Gradle 로딩이 끝날 때까지 기다립니다.

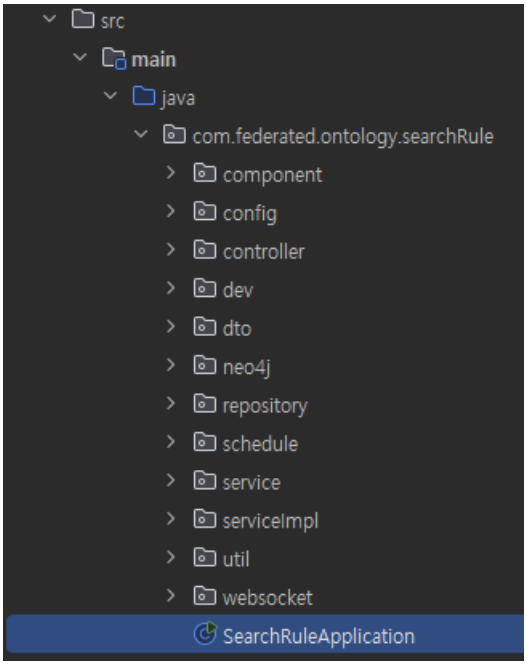
```
C:\FeDiT\OPFW\ontology_digital_twin_search\BackEnd\src\searchRule
```

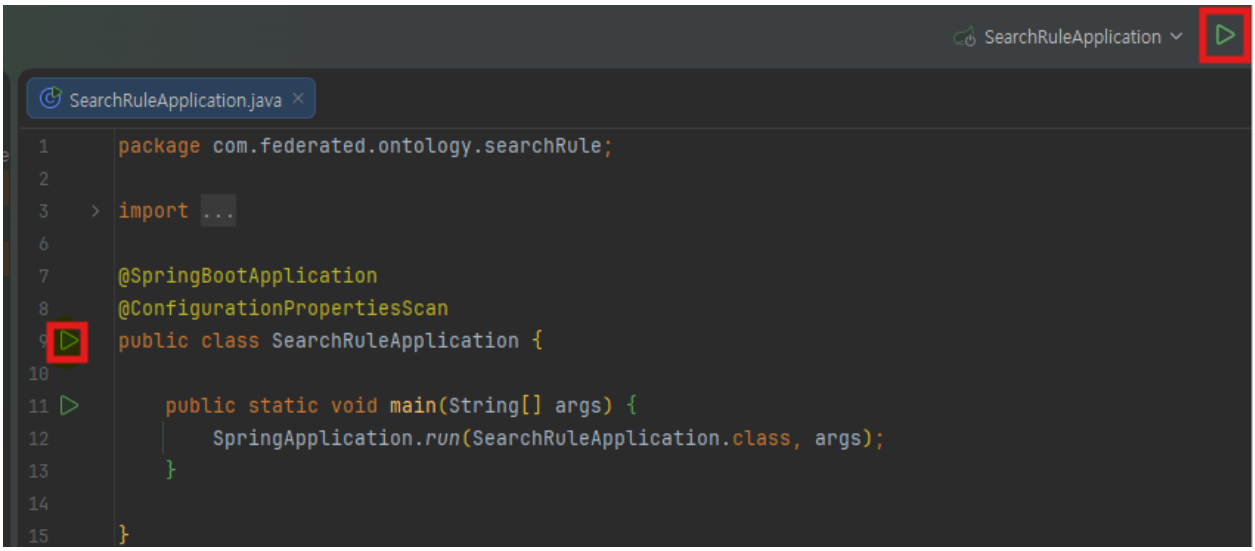
[Project Structure] → [Project] 에서 SDK 를 **JDK 17** 로 지정합니다.

Lombok 을 사용하므로 [Settings] → [Build, Execution, Deployment] → [Compiler] → [Annotation Processors] 에서 "Enable annotation processing"을 체크합니다.



[그림 3] 백엔드 프로젝트 열기 (IntelliJ)





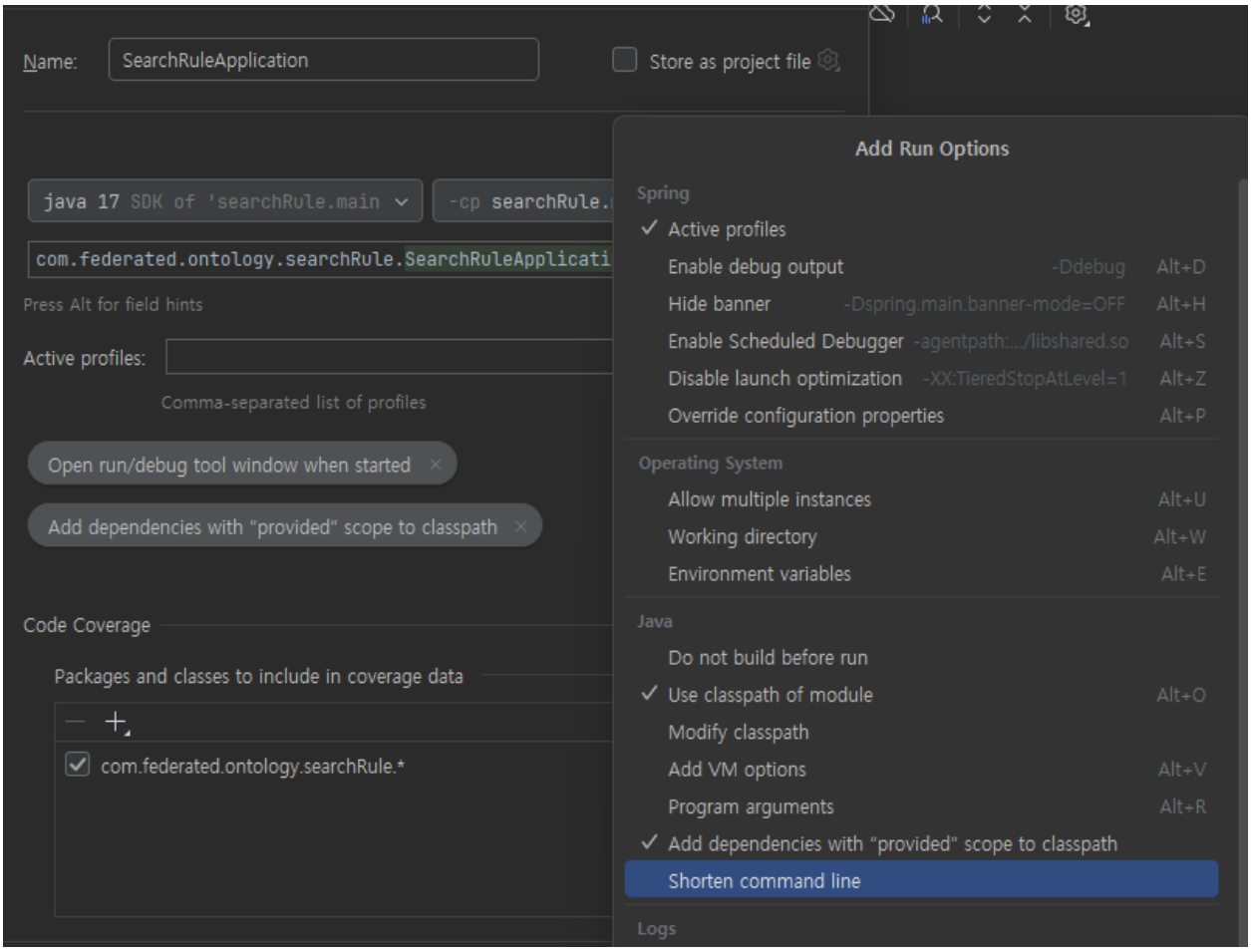
[그림 3-1] 백엔드 프로젝트 실행

프로젝트 로딩이 완료되면, 메인 클래스 SearchRuleApplication 을 열고

편집기 우측 상단 또는 좌측 라인 여백의 실행(▶) 버튼을 눌러 백엔드를 구동합니다.

콘솔에 Started SearchRuleApplication … 이 출력되고 8888 포트로 기동되면 정상입니다.

※ 만약 '**파일 이름이나 확장명이 너무 길어**' 에디터에서 실행이 불가능한 경우,
첨부한 사진을 참고하여 아래의 절차를 참고합니다.



[그림 3-2] Edit Configurations 창 — Modify Options – Shorten command line 항목 체크

1. 상단 실행버튼 옆 SearchRuleApplication 드롭다운 → Edit Configurations...
2. 왼쪽에서 SearchRuleApplication 선택
3. Modify options 클릭 → Shorten command line 항목 체크
4. 새로 생긴 드롭다운에서 JAR manifest 선택
5. OK/Apply → 다시 실행 ▸

다. 빌드 전 설정 확인

백엔드가 접속하는 외부 주소는 **빌드 시점에 JAR 안에 포함**됩니다. 배포 환경이 바뀌는 경우 아래 파일을 빌드 전에 먼저 수정해야 합니다.

```
BackEnd\src\searchRule\src\main\resources\application-docker.properties
```

항목	현재 값	설명
fedit.api.base-url	http://220.124.222.84:1219	원본 데이터(fedit) API 주소. **서버에서 접근 가능해야 하며 초기 데이터 적재에 필수**
cors.allowed-origins	http://221.133.48.18:33000,http://localhost:5173	브라우저가 백엔드를 직접 호출할 때 허용할 출처
spring.neo4j.authentication.password	test1234	.env 의 NEO4J_AUTH 와 같은 값이어야 함
spring.datasource.password	ontology1234	.env 의 OC_DB_PASS 와 같은 값이어야 함

> ※ **`cors.allowed-origins` 의 현재 값은 포트가 `33000` 입니다.** 프론트를 기본값인 8181 포트로 배포한다면 http://{serverIp}:8181 을 추가하십시오. 다만 프론트 nginx 가 /api 요청을 백엔드로 프록시하므로, 화면에서 발생하는 일반적인 호출은 동일 출처로 처리되어 CORS 가 적용되지 않습니다. 브라우저가 백엔드(8888)를 직접 호출하는 경우에만 이 설정이 필요합니다.
>
> ※ 백엔드 Dockerfile 은 --spring.profiles.active=docker 로 기동하므로 컨테이너에서는 이 파일만 적용됩니다. application.properties(dev)는 적용되지 않습니다.

라. 빌 드

○ 명령행에서 빌드하는 경우

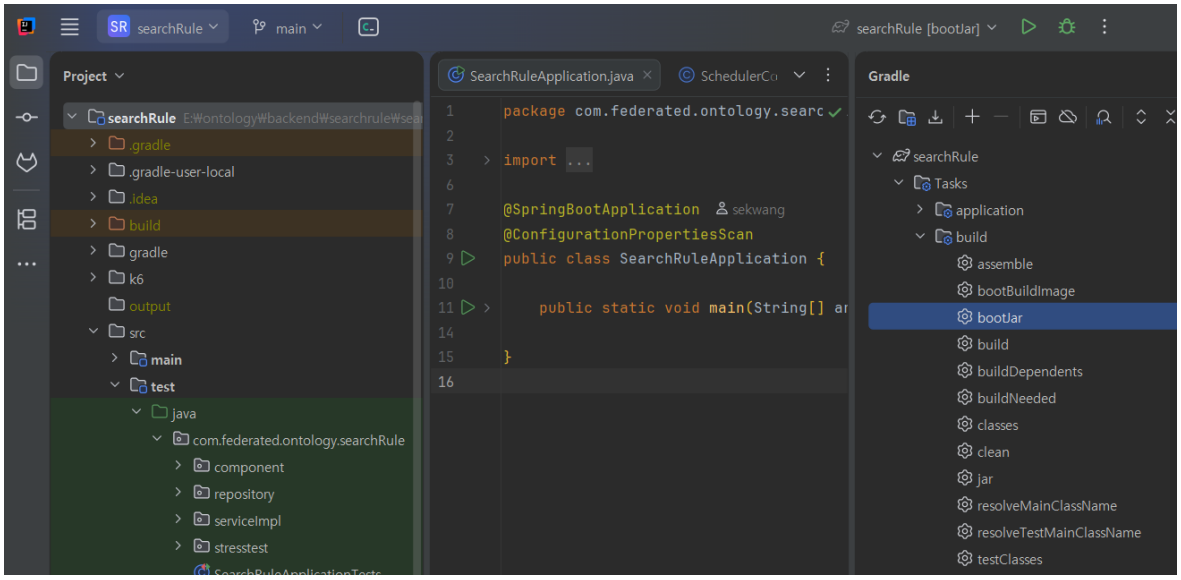
프로젝트에 Gradle 래퍼(gradlew)가 포함되어 있어 별도의 Gradle 설치가 필요하지 않습니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search\BackEnd\src\searchRule
gradlew.bat bootJar
```

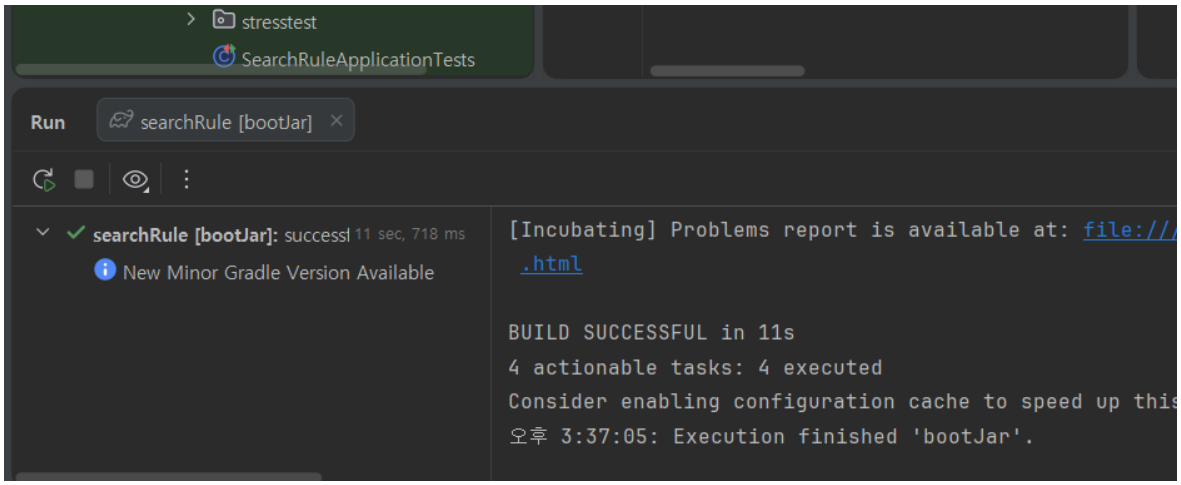
> ※ 최초 빌드 시 repo.spring.io/snapshot 에서 스냅샷 의존성을 내려받으므로 인터넷 연결이 필요하며 시간이 다소 걸릴 수 있습니다. (2 장「가」참고)

○ IntelliJ 에서 빌드하는 경우

1. 우측 Gradle 창에서 searchRule – Tasks – build – bootJar 를 선택합니다.
2. bootJar 를 더블클릭하여 실행합니다.
3. BUILD SUCCESSFUL 이 출력되면 정상 완료된 것입니다.



[그림 5] Gradle 창 — bootJar 태스크 선택

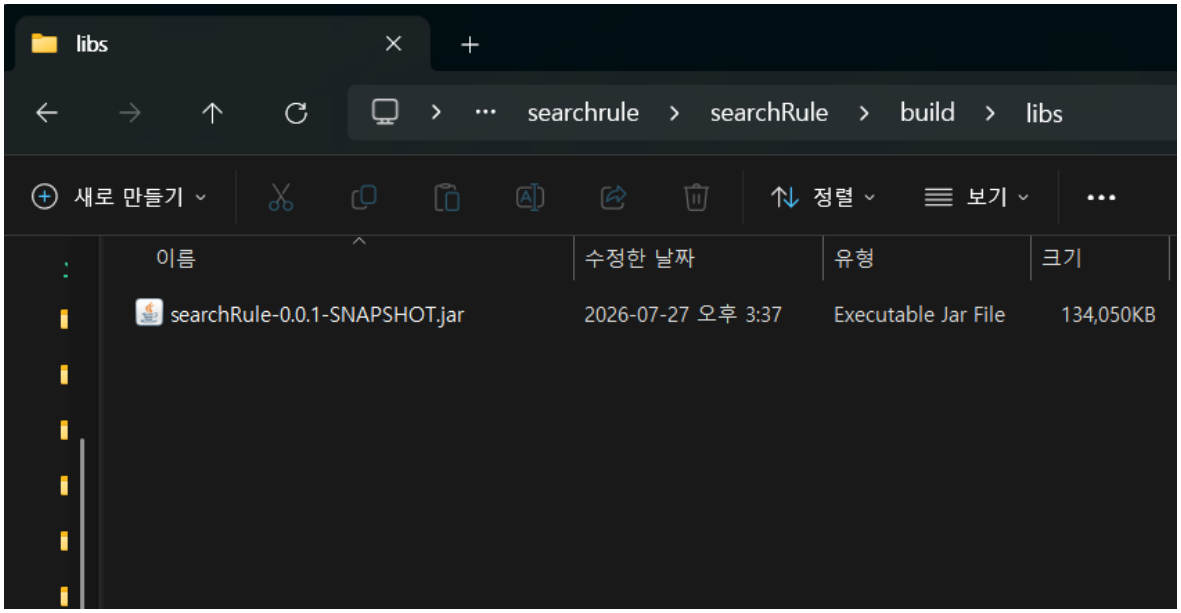


[그림 6] bootJar 실행 결과

마. 빌드 산출물 확인 및 반영

빌드가 완료되면 아래 경로에 JAR 이 생성됩니다.

```
BackEnd\src\searchRule\build\libs\searchRule-0.0.1-SNAPSHOT.jar
```



[그림 7] 빌드 산출물(JAR) 확인

생성된 JAR 을 **운용 폴더로 복사**합니다.

명령 프롬프트(CMD)에서 아래를 실행합니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search


# 1) 빌드 산출물 생성 확인 (파일 크기·시각 확인)
dir BackEnd\src\searchRule\build\libs\searchRule-0.0.1-SNAPSHOT.jar


# 2) 운용 폴더로 복사
copy /Y BackEnd\src\searchRule\build\libs\searchRule-0.0.1-SNAPSHOT.jar
initResource\OntologyConverter\


# 3) 반영 결과 확인 (시각이 방금 빌드한 시각인지 확인)
dir initResource\OntologyConverter\searchRule-0.0.1-SNAPSHOT.jar
```

> ※ JAR 내부에 어떤 설정 파일이 포함되었는지 확인하려면 아래로 목록을 볼 수 있습니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search
jar tf initResource\OntologyConverter\searchRule-0.0.1-SNAPSHOT.jar | findstr "application-
docker"
```

5. 프론트엔드

가. 프로젝트 경로 이동

명령창에서 프로젝트 폴더로 이동합니다. 이후 5 장의 모든 npm 명령은 이 폴더에서 실행합니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search\FrontEnd\src\ontology-converter-frontend
dir package.json                              # 이 파일이 보이면 올바른 폴더
node --version                                # v20.19 이상 (v22 권장)
```

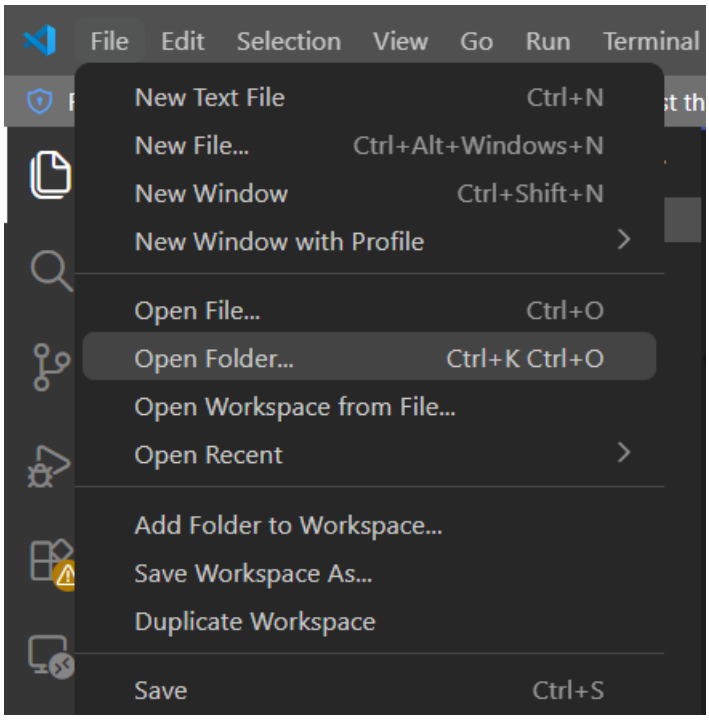
VS Code 로 열어 보려면 해당 경로에서 아래 명령을 입력합니다.

```
code .
```

○ VS Code 에서 여는 경우

Visual Studio Code 에서 [파일] → [폴더 열기] 로 아래 폴더(package.json 이 있는 폴더)를 엽니다.

```
C:\FeDiT\OPFW\ontology_digital_twin_search\FrontEnd\src\ontology-converter-frontend
```



[그림 8] 프론트엔드 프로젝트 열기 (VS Code)

나. 패키지 설치

```
npm install
```

설치가 완료되면 프로젝트 하위에 `node_modules` 폴더가 생성됩니다.

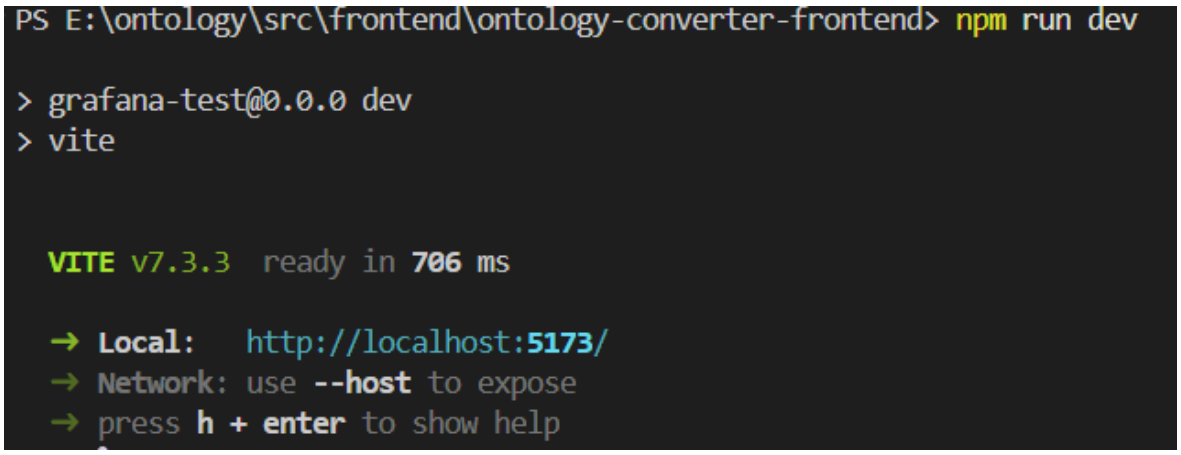
설치 완료 후 출력되는 취약점(vulnerabilities) 경고는 빌드 및 구동에 영향을 주지 않으므로, 필요 시 별도로 조치합니다.

다. 로컬 구동 확인

```
npm run dev
```

구동이 완료되면 접속 주소(`Local : http://localhost:5173/`)가 출력됩니다.

구동을 중지할 경우 터미널에서 `Ctrl + C` 를 입력합니다.



[그림 9] 프론트 로컬 구동 (npm run dev)

> ※ 그래프 화면은 Neo4j 가, 검색은 백엔드가 떠 있어야 정상 동작합니다.
> 인프라가 없으면 화면 구성만 확인됩니다.

라. 초기 데이터 적재 (최초 설치 시)

> ※ 이 절은 컨테이너를 모두 기동한 뒤에 수행합니다.

컨테이너가 모두 기동된 최초 실행 시점에는 DB(Neo4j·Stellio)가 비어 있습니다. 외부 원본(fedit API)에서 데이터를 받아 온톨로지로 변환·적재해야 화면에 데이터가 표시됩니다.

○ 실행 여부 판단

상황	조치
데이터 없음 — 신규 서버 최초 설치, 또는 볼륨을 삭제하고 재기동한 경우	아래 API 를 **1 회 실행**

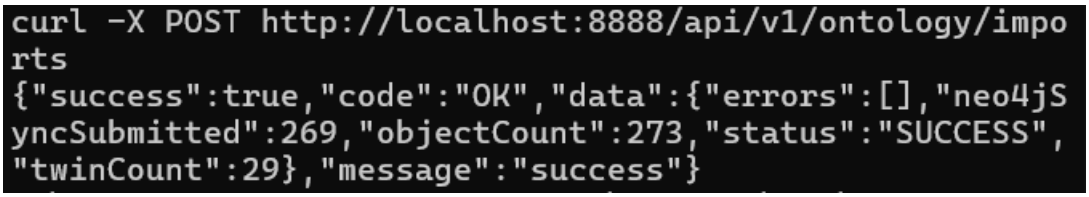
상황	조치
데이터 있음 — 운영 중인 서버에 새 버전만 재배포하는 경우	**실행하지 않습니다**

> ※ 데이터가 있는 상태에서 초기 적재 API 를 실행하면 외부 원본에서 데이터를 다시 받아 덮어쓰므로, 서버에 쌓여 있던 기존 데이터·변경분이 사라집니다. 각 데이터의 **적재 시각(`createdAt`) 이력까지 함께 초기화**됩니다.

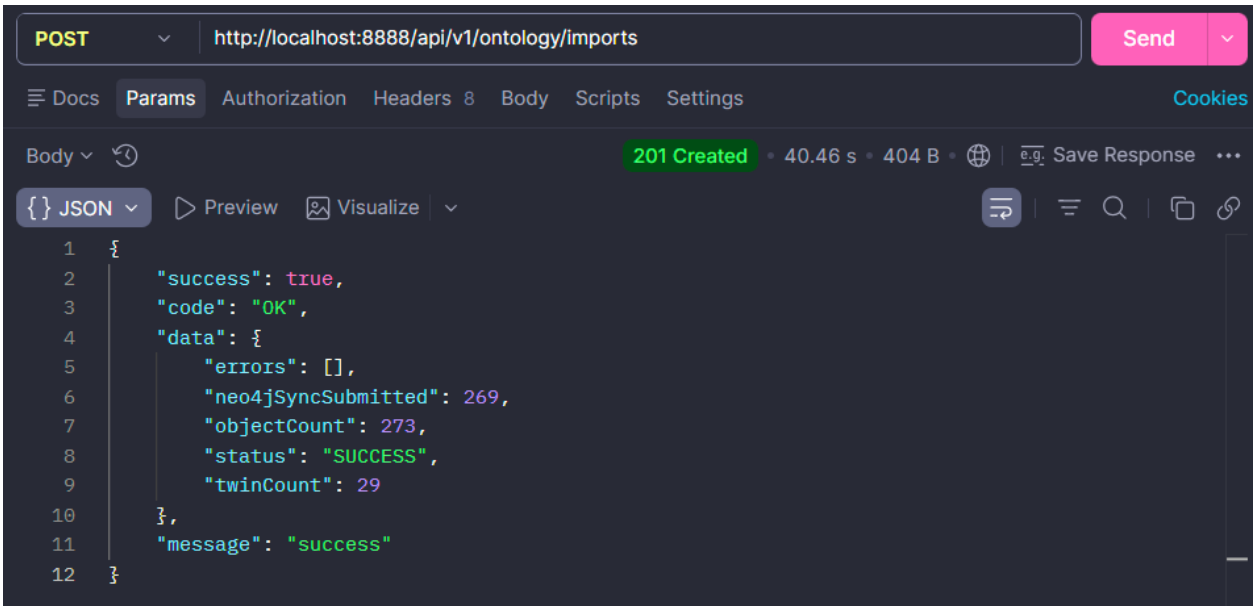
○ 초기 적재·온톨로지 생성 API

백엔드가 실행 중인 환경에서 아래 명령을 실행합니다.

```
curl -X POST http://localhost:8888/api/v1/ontology/imports   # 초기 적재 + 온톨로지 생성(초기화)
```



[그림 9-1] 초기 적재 API 실행



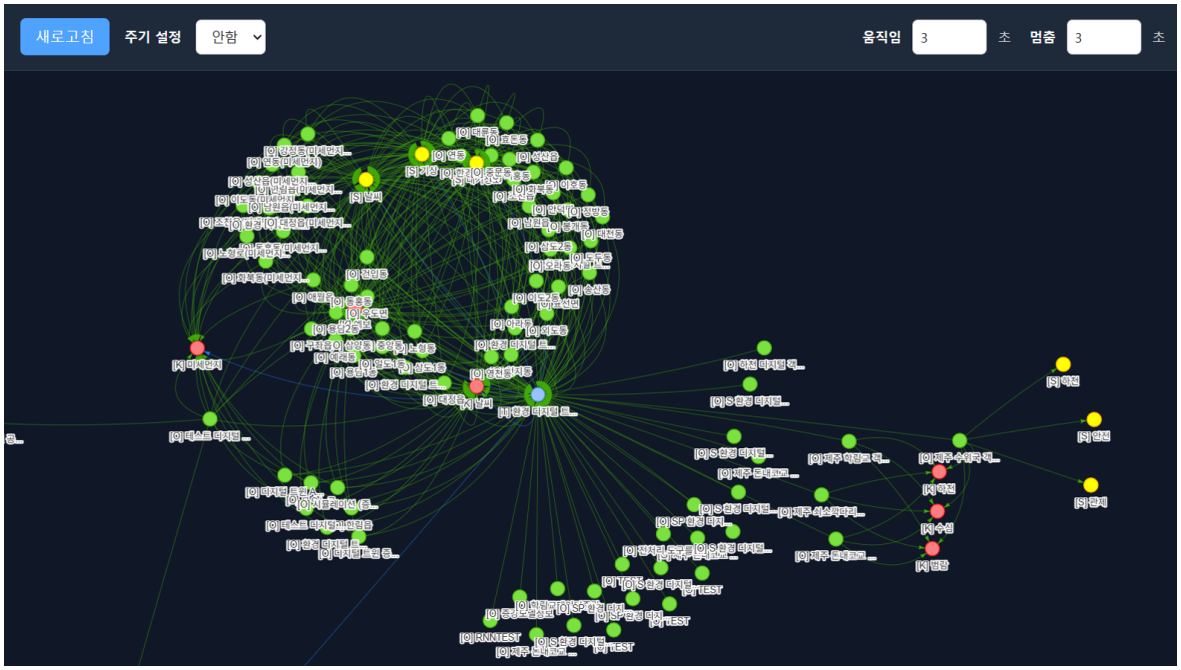
[그림 9-2] 초기 적재 API 실행 (POSTMAN 으로 호출 시)

해당 호출로 외부 원본(fedit)에서 데이터를 받아 Stellio 에 적재하고, 온톨로지를 생성하여 Neo4j 그래프까지 동기화합니다.

> ※ 8888 포트는 외부 공개 대상이 아니므로, 위 명령은 반드시 서버 내부에서 `localhost` 로 실행합니다. 외부 PC 에서 http://{serverIp}:8888 로 호출하면 접속되지 않습니다.
>
> ※ 외부에서 호출해야 한다면 프론트 nginx 의 프록시 경로를 이용합니다.
> curl -X POST http://{serverIp}:8181/ngsi-api/ontology/imports
>
> ※ fedit API 주소(`fedit.api.base-url`)가 **서버에서 접근 가능해야** 합니다. 접근 불가 시 적재가 실패합니다. (4 장「다」참고)

완료되면 Stellio 에 NGSI-LD 엔티티가, Neo4j 에 대응 그래프가 생성됩니다.

○ 제공되는 화면



[그림 10] Graph-DB 페이지

그래프 시각화 화면입니다. 데이터는 Neo4j 그래프 DB(Stellio 에서 동기화된 온톨로지)에서 가져오며, 브라우저가 Neo4j 에 직접 접속해 그립니다. 디지털 트윈·오브젝트·용어(Term) 노드와 그 사이의 관계가 표시되고, 한 노드를 선택하면 연관된 노드·관계가 함께 나타납니다.



[그림 11] Ontology-List 검색 페이지

자연어 검색 화면입니다. 검색창에 키워드를 입력하면 백엔드가 Stellio 의 NGSI-LD 엔티티와 매칭해 결과를 반환합니다. 검색어를 토큰으로 분해해 관련 오브젝트·트윈을 찾고 점수순으로 정렬하여, 추천 조합(RecommendCombination)과 추천 트윈(RecommendTwins)을 제시합니다.

마. 서버 URL 설정 (빌드 전 필수)

프론트엔드가 접속하는 주소는 **빌드 시점에 정적 파일 안에 포함**됩니다. 배포 환경이 바뀌는

경우 아래 파일을 빌드 전에 먼저 수정해야 합니다.

```
FrontEnd\src\ontology-converter-frontend\.env.production
```

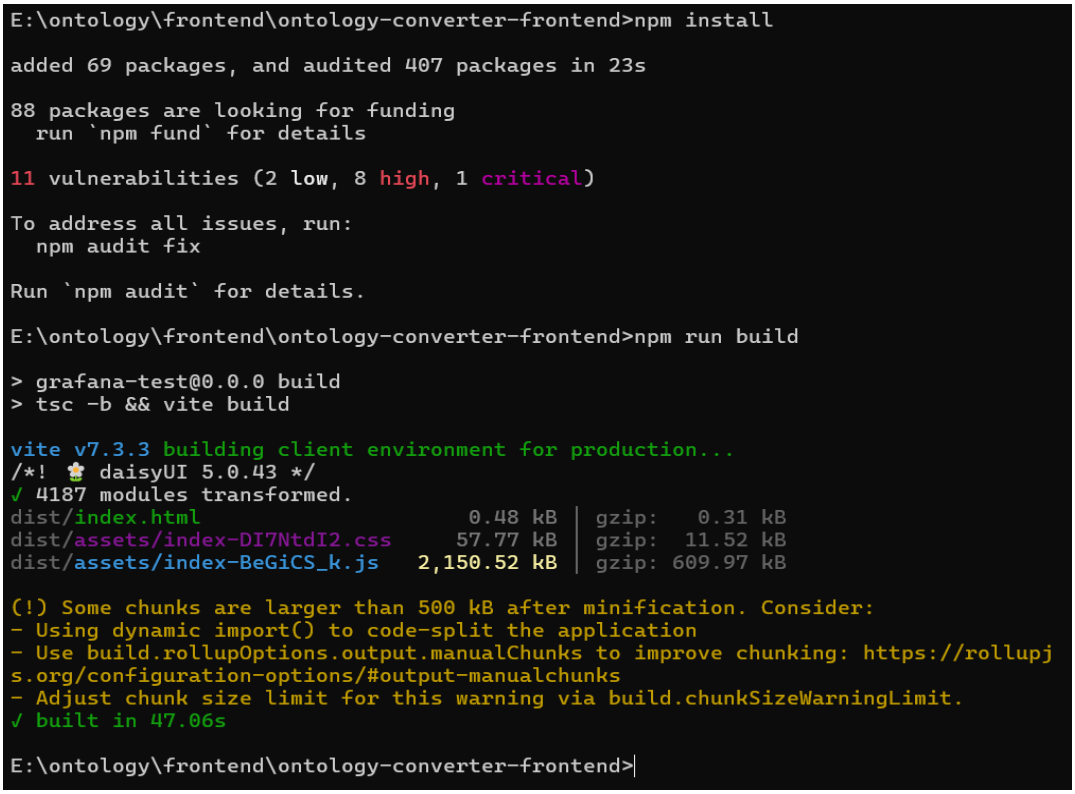
> ※ 위 값을 수정한 뒤에는 **반드시 다시 빌드**해야 반영됩니다.

변수명	현재 값	설명
VITE_NEO4J_URI	(주석 처리)	비워 두면 브라우저가 접속한 주소를 자동 사용합니다(로컬·서버 공용, 재빌드 불필요). **Neo4j 7687 포트를 바꾼 경우에만** bolt://{serverIp}:{포트} 형식으로 지정합니다
VITE_NEO4J_USER	neo4j	Neo4j 계정
VITE_NEO4J_PASSWORD	test1234	.env 의 NEO4J_AUTH 비밀번호와 일치시킵니다

바. 빌 드

```
npm run build
```

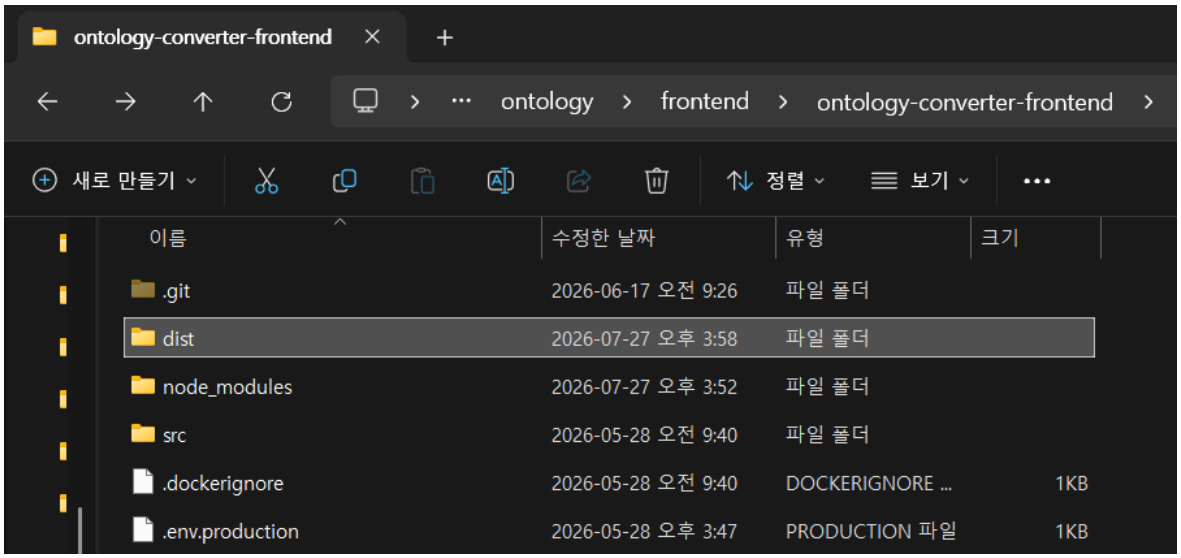
정상 완료 시 프로젝트 하위에 dist 폴더가 생성됩니다.



[그림 12] 패키지 설치 및 빌드

사. 빌드 산출물 확인 및 반영

```
FrontEnd\src\ontology-converter-frontend\dist\
 ├─ index.html
 └─ assets\
     ├─ index-{hash}.css
     └─ index-{hash}.js
```



[그림 13] 빌드 산출물(dist) 확인

생성된 dist 폴더를 **운용 폴더로 복사**합니다.

> ※ {hash} 값은 빌드할 때마다 변경되므로, 기존 dist 폴더를 지우거나 백업한 뒤 통째로 교체하는 것을 권장합니다. 이전 assets 파일이 남으면 혼동의 원인이 됩니다.

명령 프롬프트(CMD)에서 아래를 실행합니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search


# 1) 기존 dist 폴더 삭제 (이전 assets 잔존 방지)
if exist initResource\OntologyConverter\frontend\dist rmdir /s /q
initResource\OntologyConverter\frontend\dist

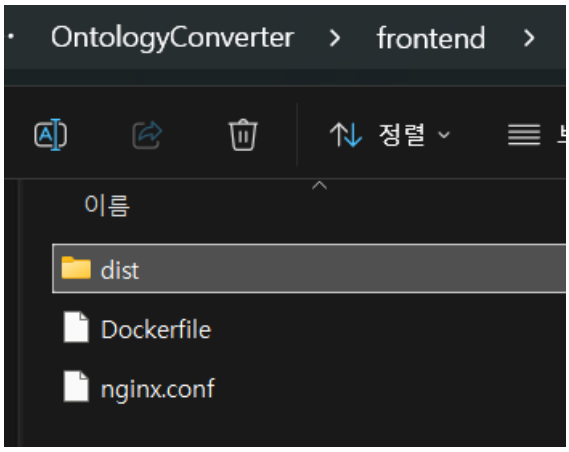

# 2) 새 dist 를 통째로 복사
xcopy FrontEnd\src\ontology-converter-frontend\dist
initResource\OntologyConverter\frontend\dist\ /E /I /Y


# 3) 반영 확인
dir initResource\OntologyConverter\frontend\dist\index.html
dir initResource\OntologyConverter\frontend\dist\assets
```

여기까지 수행하면 운용 폴더(initResource/OntologyConverter)에 최신 JAR 과 dist 가 모두 반영된 상태가 됩니다. 이후 서버 배포 절차는 6 장을 참조하십시오. (본 보완판의 명령어 추가 범위는 여기까지입니다)

배포 직전 최종 점검 명령입니다.

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search\initResource\OntologyConverter
dir
# 아래 항목이 모두 있어야 합니다
#   docker-compose.yml / .env / Dockerfile
#   searchRule-0.0.1-SNAPSHOT.jar   (4 장 산출물)
#   output\                        (빈 폴더, 없으면 mkdir output)
#   frontend\dist\index.html        (5 장 산출물)
```



[그림 14] 빌드 산출물 배치 결과 확인

6. 서버 배포

가. 이 장에서 수행하는 작업

나. 서버 접속 및 사전 준비

순서	작업	해당 절
1	서버에 접속합니다.	나
2	운용 폴더 구성을 확인합니다.	다
3	운용 폴더를 서버로 옮깁니다.	다
4	설정 파일을 운용 환경에 맞게 수정합니다.	라
5	컨테이너를 기동합니다.	마
6	초기 데이터를 적재합니다. (최초 설치 시)	5 장「라」
7	정상 기동 여부를 확인합니다.	바

본 절은 **아무것도 설치되어 있지 않고 SSH 포트만 열려 있는 Rocky Linux 9 서버**를 기준으로 합니다. 아래 명령어를 순서대로 복사하여 붙여 넣으면 배포에 필요한 실행 환경이 갖추어집니다.

> ※ 이 시점부터 6 장의 모든 명령어는 **서버에 접속한 상태**에서 입력합니다. 접속을 종료하려면 `exit` 를 입력합니다.
>
> ※ Ubuntu / Debian 계열 서버를 사용하는 경우「4) Docker 설치」와「6) SELinux 설정」만 해당 배포판의 절차로 대체하고 나머지는 동일합니다. (Ubuntu 는 SELinux 를 사용하지 않으므로 6)은 생략합니다)

○ 1) 서버 접속

작업 PC 에서 PuTTY 를 실행해 SSH 로 서버에 접속합니다. Host Name 에 {serverIp}, Port 에 {sshPort}, Connection type 은 SSH 로 지정하고, 같은 계정으로 로그인합니다. 명령 프롬프트에서 Windows 기본 ssh 명령을 사용하는 경우에는 아래 명령으로도 접속할 수 있습니다. (설치는 부록「라」 9) 참고)

```
ssh {계정}@{serverIp} -p {sshPort}
```

최초 접속 시 접속을 계속할지 묻는 메시지가 표시되면 `yes` 를 입력합니다.

이어서 비밀번호 입력을 요구하면 계정 비밀번호를 입력합니다. (입력하는 문자는 화면에 표시되지 않습니다)

접속이 완료되면 아래 명령어로 정상 접속 여부와 운영체제를 확인합니다.

```
hostname                # 서버의 이름이 출력됩니다.
whoami                  # 현재 접속한 계정이 출력됩니다.
pwd                     # 현재 위치한 경로가 출력됩니다.
cat /etc/os-release | grep -E '^(NAME|VERSION)='
```

`NAME="Rocky Linux", VERSION="9.x"` 로 출력되면 본 절의 명령어를 그대로 사용합니다.

○ 2) 시스템 사전 점검

```
uname -m                                # x86_64 로 출력되어야 합니다
free -h                                 # 메모리
df -h /                                 # 루트 디스크 여유 공간
df -h /var/lib/docker                    # 이미지·볼륨이 저장되는 경로
lsblk                                   # 디스크 및 마운트 구성
```

아래 값은 **본 시스템만 단독으로 구동할 때의 참고 기준**입니다. 서버의 실제 사양과 함께 구동되는 다른 시스템에 따라 달라지므로, 절대적인 요구 사항이 아니라 도입 검토 시의 가늠자로 활용하시기 바랍니다.

> ※ **본 시스템은 다른 연합트윈 모듈보다 자원 소요가 큽니다.** 컨테이너 9 개 중 6 개가 JVM 기반이라 메모리가 4GB 수준이면 기동 도중 일부 서비스가 종료됩니다.

항목	참고 기준	산정 근거
CPU	4 코어 이상	컨테이너 9 개 동시 구동
메모리	8GB 이상 (권장 16GB)	Stellio 3 종·온톨로지 변환기의 **Java 프로세스 4 개**에 Kafka·Neo4j(각각 JVM), PostgreSQL 2 종이 더해짐
루트 디스크 여유	40GB 이상	컨테이너 이미지 9 종 + Docker 볼륨
데이터 증가분	운용 기간에 따라 증설	Neo4j 그래프·Stellio 엔티티·TTL 산출물이 누적됨

> ※ 실제 소요량은 서비스를 기동한 뒤 `docker stats --no-stream` 으로 확인하여 조정하는 것이 가장 정확합니다. (6 장「바」이후 수행)
>
> ※ Neo4j·Kafka·PostgreSQL 데이터는 **Docker 볼륨**에 저장되며 기본 경로는 `/var/lib/docker/volumes` 입니다. 별도 디스크를 쓰려면 이 시점에 마운트를 완료해 두십시오.

○ 3) 시각 동기화

```
sudo dnf install -y chrony
sudo systemctl enable --now chronyd
sudo timedatectl set-timezone Asia/Seoul


timedatectl                             # Time zone: Asia/Seoul, System clock synchronized:
yes
chronyc sources -v | head -5
```

> ※ Kafka 컨슈머의 오프셋 처리와 Stellio 의 시계열(TimescaleDB) 저장이 서버 시각에 의존합니다. 시각이

> 어긋나면 데이터의 시간 정보가 실제와 달라집니다.

○ 4) Docker 설치

Rocky 9 의 기본 저장소에는 docker-ce 패키지가 없으므로 공식 저장소를 먼저 등록해야 합니다.

```
# (1) 충돌 패키지 확인 — podman-docker 가 있으면 docker 명령이 podman 으로 연결됩니다
rpm -q podman-docker && sudo dnf remove -y podman-docker


# (2) 구버전 docker 패키지 제거
sudo dnf remove -y docker docker-client docker-common docker-latest \
                   docker-logrotate docker-engine runc 2>/dev/null


# (3) Docker 공식 저장소 등록
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo


# (4) 설치
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
                    docker-buildx-plugin docker-compose-plugin


# (5) 기동 및 부팅 시 자동 시작 등록
sudo systemctl enable --now docker
sudo systemctl status docker --no-pager | head -5


# (6) 버전 확인
docker --version                        # Docker version 2x.x.x
docker compose version                   # Docker Compose version v2.x.x
```

> ※ docker-compose (하이픈)가 아니라 docker compose (공백)로 출력되어야 합니다. 본 문서의 모든 명령어는 플러그인 방식(docker compose)을 기준으로 합니다.
>
> ※ (5)의 enable 을 누락하면 **서버를 재부팅했을 때 Docker 가 자동으로 시작되지 않아** 컨테이너 9 개가 모두 기동되지 않습니다.
>
> ※ (3)·(4)에는 인터넷 접속이 필요합니다. 폐쇄망 서버는 7 장 「자」를 참고하십시오.

○ 5) docker 그룹 등록

docker 명령을 sudo 없이 사용할 수 있도록 현재 계정을 docker 그룹에 추가합니다.

```
sudo usermod -aG docker $USER
exit                                    # 그룹 변경은 재접속해야 적용됩니다
```

재접속한 뒤 적용 여부를 확인합니다.

```
ssh {계정}@{serverIp} -p {sshPort}


id -nG | tr ' ' '\n' | grep -x docker     # docker 가 출력되면 정상
docker ps                                 # 오류 없이 표가 출력되면 정상
```

> ※ 이 단계를 건너뛰면 이후 절의 docker 명령이 permission denied while trying to connect to the Docker daemon socket 오류로 실패합니다. 본 문서의 명령어는 모두 sudo 없이 실행하는 것을 기준으로 작성되어 있습니다.

○ 6) SELinux 설정

Rocky 9 는 SELinux 가 **Enforcing** 상태로 기본 설정되어 있습니다.

```
getenforce                                  # Enforcing · Permissive 이면 아래를 수행합니다

sudo dnf install -y policycoreutils-python-utils
sudo semanage fcontext -a -t container_file_t '/usr/local/OntologyConverter(/.*)?'
```

※ 백엔드 컨테이너는 운용 폴더의 `output/` 을 `/app/data/output` 으로 연결(bind mount)하여 TTL 산출물을 저장합니다. SELinux 컨텍스트가 맞지 않으면 **컨테이너는 기동되지만 파일 쓰기가 거부되어** 온톨로지 생성이 실패합니다.

※ Enforcing 이면 필수입니다. Disabled 는 SELinux 가 동작하지 않으므로 수행할 필요가 없습니다. Permissive 는 지금은 차단되지 않지만 이후 Enforcing 으로 전환하면 컨테이너가 output/ 에 TTL 산출물을 쓰지 못하므로, 수행해 두는 것을 권장합니다. Ubuntu / Debian 계열은 SELinux 를 사용하지 않으므로 해당하지 않습니다.

※ Neo4j·Kafka·PostgreSQL 데이터는 Docker 볼륨을 사용하므로 별도 설정이 필요하지 않습니다.

※ 위 semanage 명령은 규칙만 등록합니다. 실제 적용은 운용 폴더를 서버로 옮긴 뒤인 6 장 「다」에서 restorecon 으로 수행합니다.

※ SELinux 를 끄는 방법(`setenforce 0`)은 서버 전체의 보안 설정을 낮추므로 권장하지 않습니다.

○ 7) 포트 선점 확인

이 시스템이 사용하는 포트를 다른 서비스가 이미 쓰고 있는지 확인합니다.

```
sudo ss -lntp | grep -E ':(5432|6543|7474|7687|8083|8084|8085|8181|8888|29092) ' \
  || echo "선점된 포트 없음 — 계속 진행합니다"
```

※ 출력되는 포트가 있으면 그대로 진행할 경우 컨테이너 기동 시 `port is already allocated` 오류가 발생합니다. 기존 서비스를 중지하거나, `docker-compose.yml` 에서 해당 포트의 **왼쪽(호스트) 번호만** 변경하십시오.

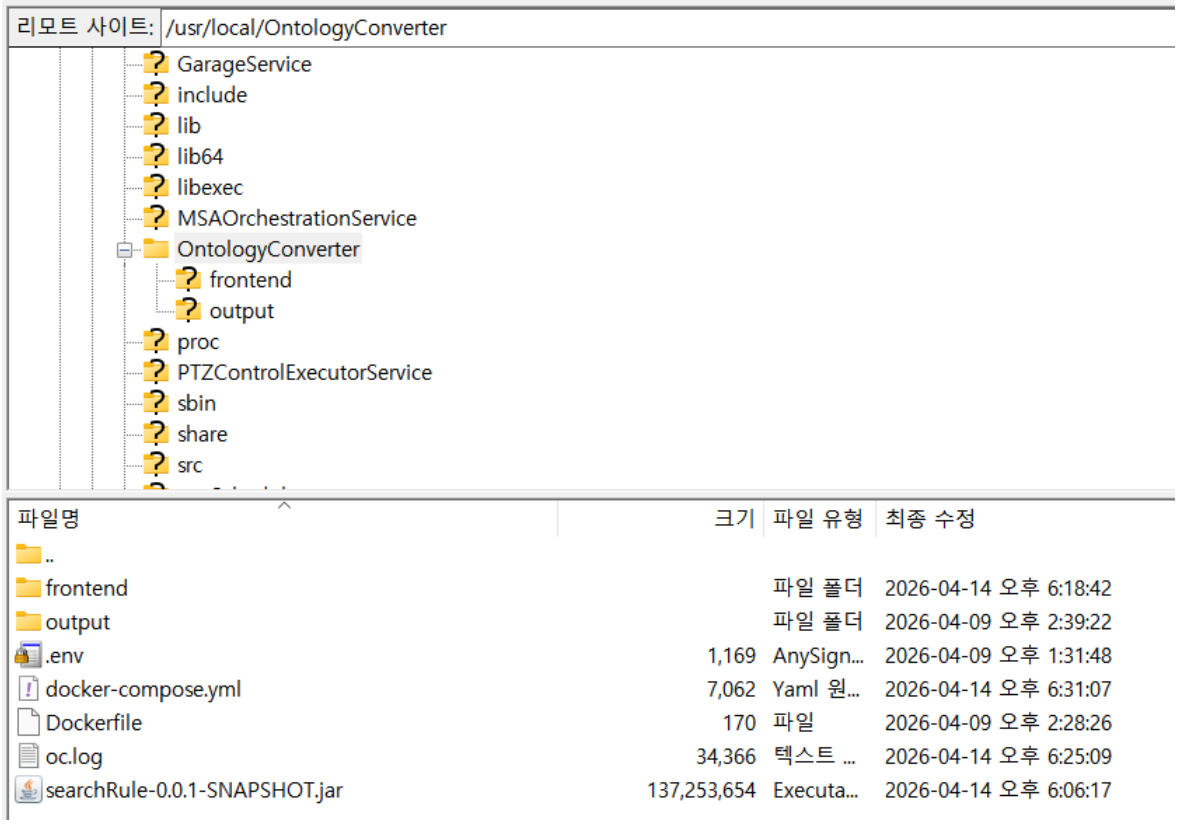
※ 본 시스템은 **5432(PostgreSQL)** 와 **7474·7687(Neo4j)** 같이 널리 쓰이는 기본 포트를 그대로 사용합니다. 서버에 이미 같은 종류의 서비스가 있으면 충돌할 가능성이 높으므로 반드시 확인하십시오.

○ 8) 설치 검증

```
docker run --rm hello-world
```

`Hello from Docker!` 가 출력되면 Docker 가 정상 동작하는 상태입니다.

※ 이 단계에서 이미지 내려받기가 실패하면 서버가 Docker Hub 에 접속하지 못하는 상태입니다. 폐쇄망 환경은 7 장 「자」, 프록시 환경은 7 장 「차」를 참고하십시오.



[그림 15] 서버 접속 및 배포 경로 확인

다. 운용 폴더 이관

파일 / 폴더	역할	준비 방법
`docker-compose.yml`	컨테이너 9 개 정의	저장소 포함
`.env`	계정·DB 명·이미지 태그	저장소 포함
`Dockerfile`	백엔드 이미지 정의 (`temurin:17-jre`)	저장소 포함
`searchRule-0.0.1-SNAPSHOT.jar`	백엔드 실행 파일	4 장 「마」에서 배치
`output/`	TTL 산출물 저장 폴더	없으면 생성
`frontend/Dockerfile`	프론트 이미지 정의 (`nginx:alpine`)	저장소 포함
`frontend/nginx.conf`	정적 서빙 + API 프록시	저장소 포함
`frontend/dist/`	화면 파일	5 장 「사」에서 배치

요청 경로	전달 대상	용도
`/`	`/usr/share/nginx/html`	화면 파일 전달 (SPA 라우팅 포함)
`/api/`	`http://ontology-converter:8888`	백엔드 API
`/ngsi-api/`	`http://ontology-converter:8888/api/v1/`	프론트의 `/ngsi-api/*` 를 백엔드 `/api/v1/*` 로 재작성하여 전달

ontology_initResource.zip 을 서버의 홈 디렉터리로 업로드합니다. (작업 PC 에서 입력)

```
cd /d C:\FeDiT\OPFW\ontology_digital_twin_search

scp -P {sshPort} ontology_initResource.zip {계정}@{serverIp}:~/
```

WinSCP 를 사용하는 경우, 아래와 같이 접속한 뒤 ontology_initResource.zip 파일을 홈 디렉터리로 끌어다 놓거나 우클릭 - [업로드] 를 선택합니다. 업로드 후 압축 해제는 위 2)와

동일하게 서버에서 수행합니다.

프로토콜을 SFTP - SSH File Transfer Protocol 로 지정합니다. (FTP 를 고르면 접속되지 않습니다. SFTP 는 SSH 위에서 동작하므로 서버에 추가 설치가 필요 없습니다)

호스트에 {serverIp}, 포트에 {sshPort} 를 입력합니다. 위「나」1) 서버 접속에서 사용한 값과 동일합니다.

ontology_initResource.zip 파일 하나만 홈 디렉터리로 업로드합니다. zip 안에 .env 등 숨김 파일이 모두 포함되어 있으므로 숨김 파일 표시 설정이 필요 없습니다.

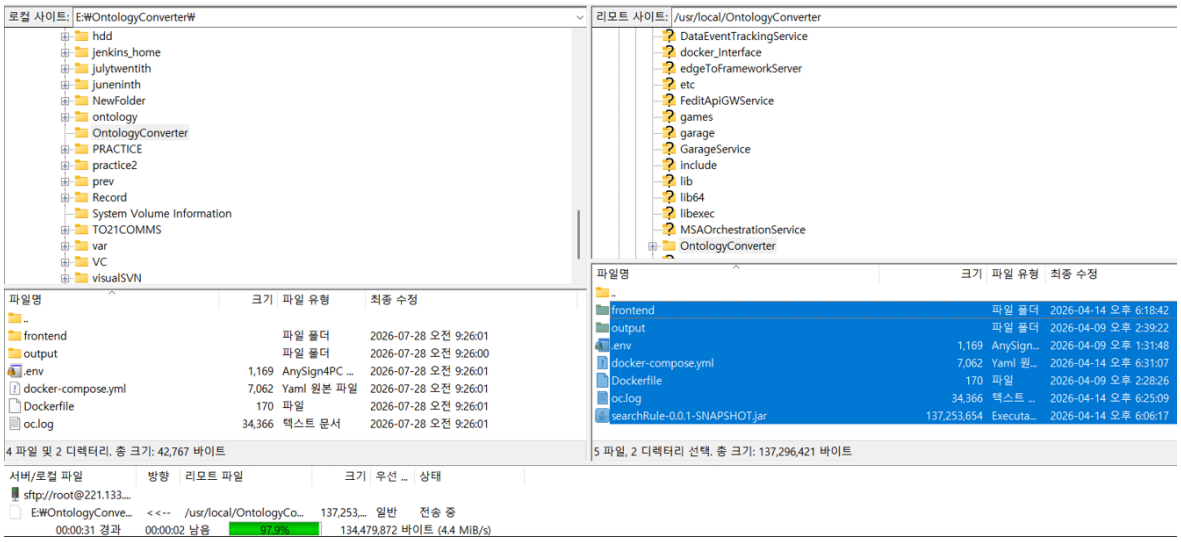
서버에서 압축을 해제합니다. (서버에서 입력)

```
sudo dnf install -y unzip          # unzip 이 설치되어 있지 않은 경우에만
cd ~
unzip -q ontology_initResource.zip
rm -rf __MACOSX                    # macOS 에서 압축한 경우에만 생성됩니다
ls -al ~/initResource/OntologyConverter
```

서버의 운용 경로로 복사하고 권한을 설정합니다. (서버에서 입력)

```
sudo cp -r ~/initResource/OntologyConverter /usr/local/OntologyConverter
sudo mkdir -p /usr/local/OntologyConverter/output      # TTL 산출물 저장 폴더 (zip 에 포함되지 않음)
sudo chown -R $USER:$USER /usr/local/OntologyConverter
sudo chmod -R 755 /usr/local/OntologyConverter
```

> ※ /usr/local 은 root 소유이므로 일반 계정으로 직접 업로드하면 Permission denied 가 발생합니다. 반드시 홈 디렉터리로 업로드한 뒤, 서버에 접속해 아래 명령으로 운용 경로에 복사하십시오.
>
> ※ cp -r 를 사용하므로 홈 디렉터리의 initResource 원본이 그대로 남습니다. 재이관이나 설정 파일 비교에 사용할 수 있으며, 필요 없으면 rm -rf ~/initResource ~/ontology_initResource.zip 로 정리하십시오. 또한 /usr/local/OntologyConverter 가 이미 존재하면 그 안쪽으로 복사되므로, 재설치 시에는 기존 폴더를 먼저 이름 변경하거나 삭제하십시오.



[그림 17] 운용 폴더 업로드

업로드 결과를 확인합니다. (서버에서 입력)

```
cd /usr/local/OntologyConverter
ls -al                                 # .env 포함 확인
ls -l searchRule-0.0.1-SNAPSHOT.jar    # 백엔드 JAR
```

```
ls -l frontend/dist/index.html       # 프론트 dist
ls -ld output                       # TTL 출력 폴더
```

※ **`.env` 는 이름이 점(`.`)으로 시작하는 숨김 파일입니다.** 와일드카드(`./OntologyConverter/*`)로 복사하면 누락되므로 반드시 폴더 전체를 복사하십시오. FTP 클라이언트에서 보이지 않으면 [서버] – [강제로 숨김 파일 보이기] 옵션을 활성화합니다.

※ **운용 폴더 이름을 바꾸지 마십시오.** Docker Compose 는 폴더 이름을 프로젝트명으로 사용하며 볼륨 이름도 여기에 묶입니다. 이름이 바뀌면 빈 볼륨이 새로 생성되어 기존 데이터를 잃습니다.

○ SELinux 컨텍스트 적용 (Rocky / RHEL 계열 필수)

6 장「나」6)에서 등록한 규칙을 방금 업로드한 운용 폴더에 실제로 적용합니다.

```
getenforce                              # Disabled 이면 아래 restorecon 은 생략
sudo restorecon -Rv /usr/local/OntologyConverter

ls -Z /usr/local/OntologyConverter       # container_file_t 로 표시되면 정상
ls -Zd /usr/local/OntologyConverter/output
```

※ 이 단계를 건너뛰면 백엔드 컨테이너가 기동되어도 TTL 파일을 쓰지 못해 온톨로지 생성이 실패합니다. 6 장「아」의 "TTL 파일 생성 실패" 항목도 함께 참고하십시오.

※ Enforcing 이면 restorecon 을 반드시 수행해야 합니다. Disabled 는 수행할 필요가 없습니다. Permissive 는 지금 수행하지 않아도 동작하지만, 이후 Enforcing 으로 전환하면 컨테이너가 TTL 산출물을 쓰지 못하므로 수행해 두는 것을 권장합니다. Ubuntu / Debian 계열은 해당하지 않습니다. SELinux 상태 확인과 규칙 등록 절차는 6 장「나」○ 6) SELinux 설정 을 참고하십시오.

○ 요청 전달(프록시) 설정

`frontend/nginx.conf` 는 아래와 같이 요청을 전달합니다.

nginx.conf

E: > ontology > docker > frontend > nginx.conf

```
server {
  listen 80;

  location / {
    root /usr/share/nginx/html;
    index index.html;
    try_files $uri /index.html;
  }

  location /api/ {
    proxy_pass http://ontology-converter:8888;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }

  # 프론트엔드가 호출하는 /ngsi-api/* 를 백엔드 /api/v1/* 로 프록시
  # (vite dev 서버의 /ngsi-api 프록시 규칙을 운영 nginx에서도 동일하게 재현)
  location /ngsi-api/ {
    rewrite ^/ngsi-api/(.*)$ /api/v1/$1 break;
    proxy_pass http://ontology-converter:8888;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }
}
```

[그림 16] nginx.conf 내부

라. 설정 파일 수정

.env 는 compose 가 각 컨테이너에 주입하는 계정·DB 명·이미지 태그를 모아둔 파일입니다.

```
8   POSTGRES_USER=stellio
9   POSTGRES_PASS=stellio_password
```

[그림 18] .env 내부

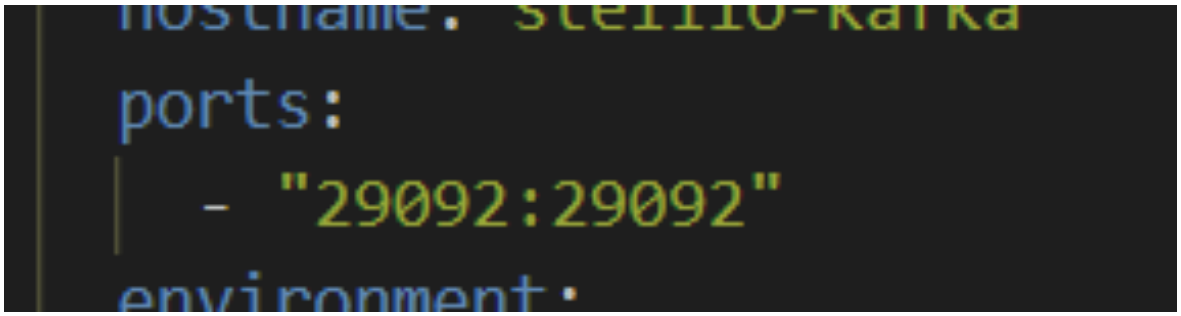
변수	현재 값	설명
STELLIO_DOCKER_TAG	latest-dev	Stellio 이미지 태그
ENVIRONMENT	docker	Stellio Spring 프로필
POSTGRES_USER / POSTGRES_PASS	stellio / stellio_password	Stellio PostgreSQL 계정
STELLIO_SEARCH_DB_DATABASE	stellio_search	Stellio 검색 DB
STELLIO_SUBSCRIPTION_DB_DATABASE	stellio_subscription	Stellio 구독 DB
NEO4J_AUTH	neo4j/test1234	Neo4j 계정
OC_DB / OC_DB_USER / OC_DB_PASS	ontology_converter / ontology_converter / ontology1234	변환기 PostgreSQL

> ※ **비밀번호를 바꾸면 백엔드·프론트도 같은 값을 알아야 합니다.**
> 둘 다 소스에 들어 있으므로 4 장 「다」·5 장 「마」에서 수정한 뒤 재빌드해야 합니다.
>
> ※ .env 변경 - 함께 바꿔야 할 곳
> 1. NEO4J_AUTH 비밀번호 - 백엔드 spring.neo4j.authentication.password · 프론트

> VITE_NEO4J_PASSWORD
> 2. OC_DB_PASS - 백엔드 spring.datasource.password
>
> ※ **운용 배포 전 자격 증명을 교체하세요.** 저장소에 포함된 값은 초기값입니다.

○ 포트를 변경해야 하는 경우

docker-compose.yml 의 ports: "호스트:컨테이너" 형식에서 **왼쪽(호스트) 숫자만** 바꾸고 오른쪽은 그대로 둡니다.



[그림 19] docker-compose.yml — 포트 설정 부분

서비스	포트	외부 공개	바꾸면 함께 수정
front-end	8181:80	공개	CORS(4 장「다」)
ontology-converter	8888:8888	내부	내부 프록시는 컨테이너명 기준이라 무관
neo4j	7474 · 7687	공개	VITE_NEO4J_URI(5 장 「마」)
api-gateway (Stellio)	8085:8080	선택	dev 프로필 stellio.broker.base-url
search · subscription	8083 · 8084	내부	—
PostgreSQL (2 종)	5432 · 6543	내부	dev 프로필 spring.datasource.url
kafka	29092:29092	내부	dev 프로필 spring.kafka.bootstrap-servers

마. 컨테이너 기동

```
cd /usr/local/OntologyConverter
docker compose up -d --build
```

--build 는 docker-compose.yml 에 build: 로 선언된 백엔드·프론트 이미지를 새로 만드는 옵션입니다.

최초 기동 시 이미지 다운로드로 수 분 소요됩니다.

JAR·dist 배치까지 통합폴더에 정상적으로 반영되었다면,

배포 전 작업 PC 내 initResource/OntologyConverter/ 폴더에서 동일하게 docker compose up -d --build 로 9 개 컨테이너 통합 구동을 미리 확인할 수 있습니다.

이를 통해 배포 전 통합폴더의 구성이 완료된 후 백엔드나 프론트, 인프라들이 한 번의 docker compose up –d 로 정상 구동 가능한지 직접 실행하고 확인해보는 과정이 필요합니다.

> ※ **JAR 이나 dist 를 교체한 뒤에는 반드시 `--build` 를 붙여야 합니다.** 없으면 기존 이미지가 그대로 재사용되어 변경 사항이 반영되지 않습니다.

```
docker compose ps        # 컨테이너 9 개가 모두 Up 인지 확인
```

> ※ DB·Kafka 가 먼저 기동된 뒤 백엔드가 연결되므로, 최초 기동 직후 백엔드가 잠깐 재시작될 수 있으나 의존 서비스가 준비되면 안정화됩니다. (`restart: on-failure`)

바. 구동 상태 확인

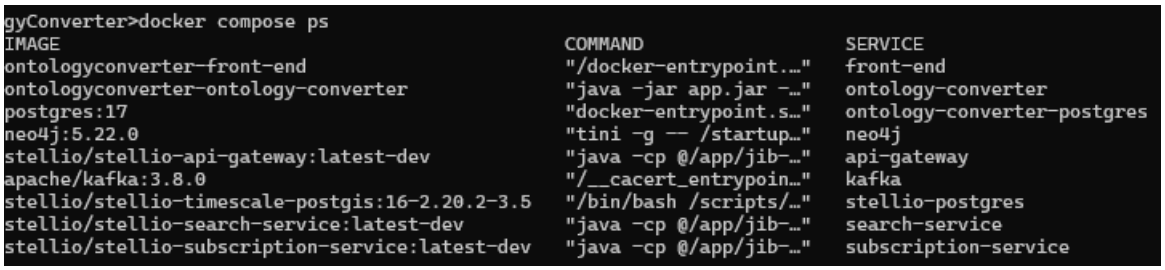
환경	추가 설정	확인 대상
사내망 / 공유기 환경	8181·7474·7687 포트 포워딩 설정	네트워크 담당자
클라우드 환경 (AWS, Azure, NCP 등)	보안 그룹(Security Group) 인바운드 규칙 허용	클라우드 관리자

```
docker compose ps
```

STATUS 열이 모두 Up(헬스체크가 있는 DB 는 Up (healthy))인지 확인합니다.

아래 9 개 컨테이너가 모두 표시되어야 합니다.

```
front-end        ontology-converter          ontology-neo4j
stellio-kafka    stellio-postgres            stellio-api-gateway
stellio-search-service    stellio-subscription-service
ontology-converter-postgres
```



[그림 21] 구동 컨테이너 9 개 확인

○ 로그 확인

```
docker compose logs -f ontology-converter    # 백엔드 로그
docker compose logs -f front-end            # 웹 서버 로그
```

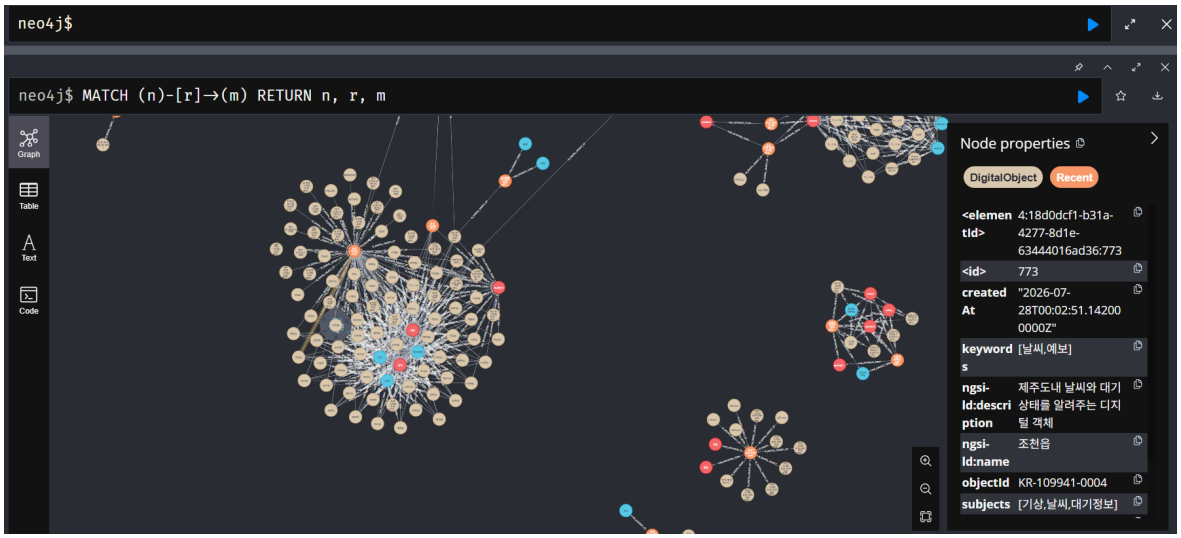
○ 서버 내부 응답 확인

```
curl -I http://localhost:8181/                  # 프론트 화면
curl -s http://localhost:8888/actuator/health    # 백엔드 상태
curl -s http://localhost:8083/actuator/health    # Stellio search service
```

○ Neo4j 데이터 확인

브라우저에서 `http://{serverIp}:7474` 로 접속하고 계정(`neo4j` / `test1234`)으로 로그인한 뒤, 상단 쿼리 입력창에 아래 쿼리를 실행합니다.

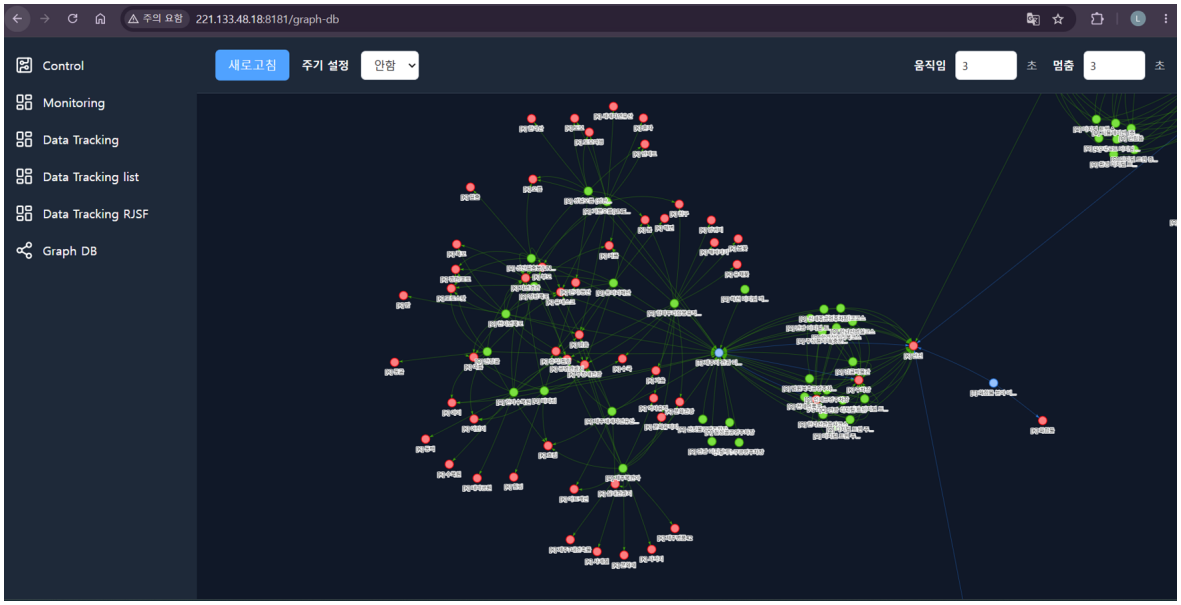
```
MATCH (n)-[r]->(m) RETURN n, r, m
```



[그림 22] Neo4j Browser 데이터 조회

○ 웹 브라우저 확인 주소

구분	접속 주소
검색·그래프 화면	`http://{serverIp}:8181/`
Neo4j Browser	`http://{serverIp}:7474`



[그림 23] 배포 결과 확인 (웹 화면)

○ 배포 확인 체크리스트

No	확인 항목	확인 내용
1	백엔드 빌드	JAR 생성 및 운용 폴더 반영 여부
2	프론트 빌드	URL 수정 후 `npm run build` 재수행 및 `dist` 반영 여부
3	숨김 파일 전송	서버의 `.env` 존재 여부

No	확인 항목	확인 내용
4	output 폴더	`output/` 존재 및 쓰기 권한 여부
5	이미지 재빌드	산출물 교체 후 `--build` 옵션 사용 여부
6	컨테이너 기동	`docker compose ps` 에 9 개 컨테이너 모두 `Up` 표시 여부
7	초기 적재	최초 설치 시 import API 실행 여부 (재배포 시에는 실행 금지)
8	화면 표시	`http://{serverIp}:8181` 접속 시 화면 정상 표시 여부
9	그래프 연동	Neo4j Browser 및 화면 그래프 데이터 표시 여부
10	외부 접근	8181 · 7687 · 7474 에 이용자 PC 에서 접근 가능한지 여부

사. 컨테이너 중지 및 재기동

구분	명령어	설명
기동	`docker compose up -d --build`	이미지를 만들고 컨테이너를 기동합니다.
중지	`docker compose stop`	컨테이너만 중지합니다. (설정 / 데이터 유지)
재개	`docker compose start`	중지된 컨테이너를 다시 기동합니다.
재기동	`docker compose restart {서비스명}`	특정 서비스를 재시작합니다.
삭제	`docker compose down`	컨테이너를 중지·삭제합니다. **볼륨(데이터)은 유지됩니다.**
상태 확인	`docker compose ps`	컨테이너 목록·상태 확인
로그 확인	`docker compose logs -f {서비스명}`	실시간 로그 확인

○ 새 버전 배포 시

운용 폴더 이름을 그대로 유지한 채(같은 볼륨 재사용) 폴더 안의 산출물만 교체합니다.

```
cd /usr/local/OntologyConverter
docker compose down           # 컨테이너만 내림, 볼륨(데이터)은 유지 → 안전
docker compose up -d --build   # 새 이미지로 재빌드·기동
```

이 순서에서는 데이터가 볼륨에 그대로 남으므로 **초기 적재 API 를 호출하지 않습니다.**

> ※ **`docker compose down -v` 사용 주의** — -v 옵션은 볼륨(데이터)까지 삭제하므로 Neo4j·DB 의 모든 데이터가 사라집니다. 삭제했다면 초기 적재 API 를 다시 실행해야 하며, 이 경우 `createdAt` 이 리셋됩니다.

아. 문제 해결

증상	원인	조치
백엔드 빌드 실패 (의존성 해석 오류)	Spring Boot 스냅샷 저장소 접근 불가	인터넷 연결 확인. 폐쇄망이면 사내 저장소에 의존성 캐시 필요 (2 장 「가」)
프론트 빌드 실패	Node.js 버전이 20.19 미만	Node 22 LTS 설치 후 재시도

증상	원인	조치
컨테이너 기동 시 포트 충돌	동일 포트를 사용하는 다른 프로그램 구동 중	`sudo lsof -i :{포트}` 확인 후 compose 의 호스트 포트 변경
백엔드가 계속 재시작됨	DB·Kafka 준비 전 연결 시도	수십 초 대기 후 재확인. 지속되면 `docker compose logs -f ontology-converter` 확인
백엔드 DB 접속 실패	`.env` 와 `application-docker.properties` 의 계정 불일치	양쪽 값 확인 (6 장「라」)
소스를 고쳤는데 반영되지 않음	`--build` 없이 기동하여 기존 이미지 재사용	`docker compose up -d --build` 로 재기동
화면은 뜨는데 데이터가 없음	초기 데이터 미적재	5 장「라」의 import API 실행
초기 적재 API 가 실패함	fedit API 주소에 서버가 접근 불가	`fedit.api.base-url` 확인 (4 장「다」)
TTL 파일 생성 실패	`output/` 폴더 없음 또는 권한 부족	`sudo mkdir -p /usr/local/OntologyConverter/output` 후 권한 확인 (6 장 「다」)
그래프 화면이 비어 있음	7687(Bolt) 포트 미개방 또는 `VITE_NEO4J_URI` 불일치	7687 포트 접근 확인 및 프론트 재빌드 (5 장「마」)
브라우저에서 API 호출이 차단됨	CORS 설정에 프론트 주소 누락	`cors.allowed-origins` 확인 후 백엔드 재빌드 (4 장「다」)
재배포 후 데이터가 사라짐	운용 폴더 이름 변경으로 새 볼륨 생성	폴더 이름을 원래대로 되돌린 뒤 재기동
Windows 에서 Clone 실패 (`Filename too long`)	경로 길이 제한(260 자) 초과	`git config --global core.longpaths true` 설정 후 재시도
`docker` 명령이 `permission denied … docker.sock` 오류	계정이 `docker` 그룹에 없음	6 장「나」 5) 수행 후 **재접속**
`docker: command not found`	Docker 가 설치되지 않음	6 장「나」 4) 수행
재부팅 후 컨테이너가 하나도 없음	Docker 서비스가 자동 시작되지 않음	`sudo systemctl enable --now docker` (6 장「나」 4) (5))
`docker pull` / `dnf install` 이 응답 없음	인터넷 차단 또는 프록시 미설정	7 장「자」(폐쇄망) · 「차」(프록시) 참고
시계열 데이터의 시각이 실제와 다름	서버 시각 미동기화 또는 시간대 불일치	6 장「나」 3) 수행 (`timedatectl`)

7. 부록

가. 신규 서버 구축 순서 요약

순서	작업	명령어 / 참조	위치
1	소스 코드 내려받기	`git clone https://github.com/etri/FeDiT.git`	작업 PC
2	백엔드 설정 수정	4 장「다」	작업 PC
3	백엔드 빌드	`gradlew bootJar`	작업 PC
4	프론트 설정 수정	5 장「마」	작업 PC
5	프론트 빌드	`npm install → npm run build`	작업 PC
6	산출물 배치	4 장「마」· 5 장「사」	작업 PC
7	운용 폴더 이관	`scp -r ./OntologyConverter …`	작업 PC
8	`.env` 수정	6 장「라」	서버
9	컨테이너 기동	`docker compose up -d --build`	서버
10	초기 데이터 적재	5 장「라」(최초 1 회)	서버
11	구동 확인	`http://{serverIp}:8181/`	브라우저

나. 백엔드 갱신 배포 순서 요약

순서	작업	명령어 / 참조	위치
1	최신 소스 반영	`git pull origin main`	작업 PC
2	빌드 전 설정 확인	4 장「다」	작업 PC
3	백엔드 빌드	`gradlew bootJar`	작업 PC
4	JAR 을 운용 폴더에 반영	4 장「마」	작업 PC
5	산출물 업로드	`scp` (JAR)	작업 PC
6	재빌드·기동	`docker compose up -d --build ontology-converter`	서버
7	구동 확인	`docker compose ps`	서버

> ※ 갱신 배포에서는 **초기 적재 API 를 실행하지 않습니다.**

다. 화면 갱신 배포 순서 요약

> ※ 화면만 교체하는 경우 백엔드 이미지는 다시 만들지 않아도 됩니다. 브라우저에 이전 화면이 남아 있으면 강력 새로 고침(`Ctrl + Shift + R`)으로 캐시를 지우고 확인하십시오.

라. 개발·배포 도구 설치 상세

순서	작업	명령어 / 참조	위치
1	최신 소스 반영	`git pull origin main`	작업 PC
2	서버 URL 설정 확인	5 장「마」	작업 PC
3	프론트 빌드	`npm run build`	작업 PC

순서	작업	명령어 / 참조	위치
4	dist 를 운용 폴더에 반영	5 장「사」	작업 PC
5	산출물 업로드	scp (frontend/dist)	작업 PC
6	재빌드·기동	`docker compose up -d --build front-end`	서버
7	화면 확인 (캐시 삭제 후)	`http://{serverIp}:8181/`	브라우저

○ winget 사용 준비

아래 설치 명령은 모두 관리자 권한 명령 프롬프트에서 실행합니다. (여는 방법은 2 장「라」의「○ 명령 프롬프트 여는 방법」참고)

> ※ winget 은 Windows 11 에 기본 포함이며, Windows 10 도 1809(빌드 17763) 이상이면 사용할 수 있습니다. winget 명령이 없다면「앱 설치 관리자(App Installer)」가 없거나 오래된 것이므로 Microsoft Store 에서 설치·업데이트하십시오. Store 를 사용할 수 없는 경우 아래와 같이 설치 패키지를 직접 내려받아 등록할 수 있습니다.

```
curl.exe -L -o "%USERPROFILE%\Downloads\winget.msixbundle" https://aka.ms/getwinget
winget --version
```

1) Git

○ 명령어로 설치

```
winget install -e --id Git.Git --accept-package-agreements --accept-source-agreements
```

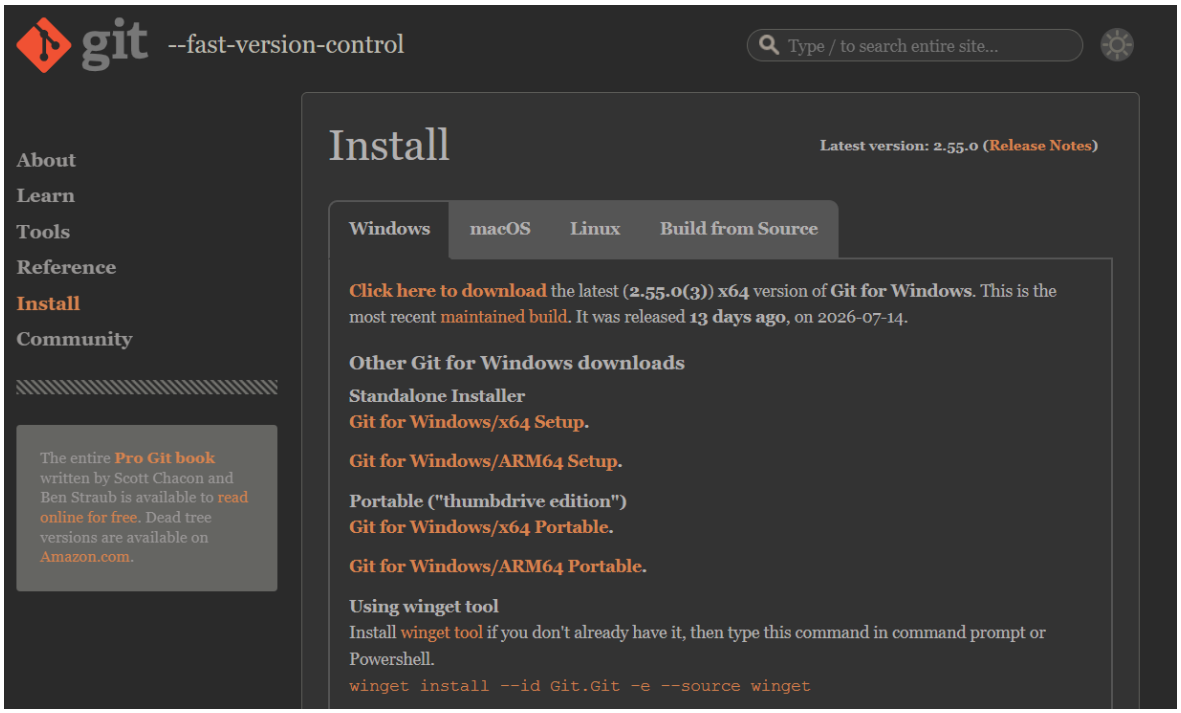
설치 후 새 명령창을 열어 확인합니다. (원본의 PATH·줄바꿈 옵션은 winget 설치 시 기본값으로 적용됩니다)

설치가 완료되었다면 버전을 확인하고, 오류 방지를 위한 Git 의 긴 경로지원, 전역 Git 설정을 아래 내용 입력을 통해 활성화한다

```
git --version
git config --global core.longpaths true
git config --global --list
```

○ 다운로드 페이지에서 설치

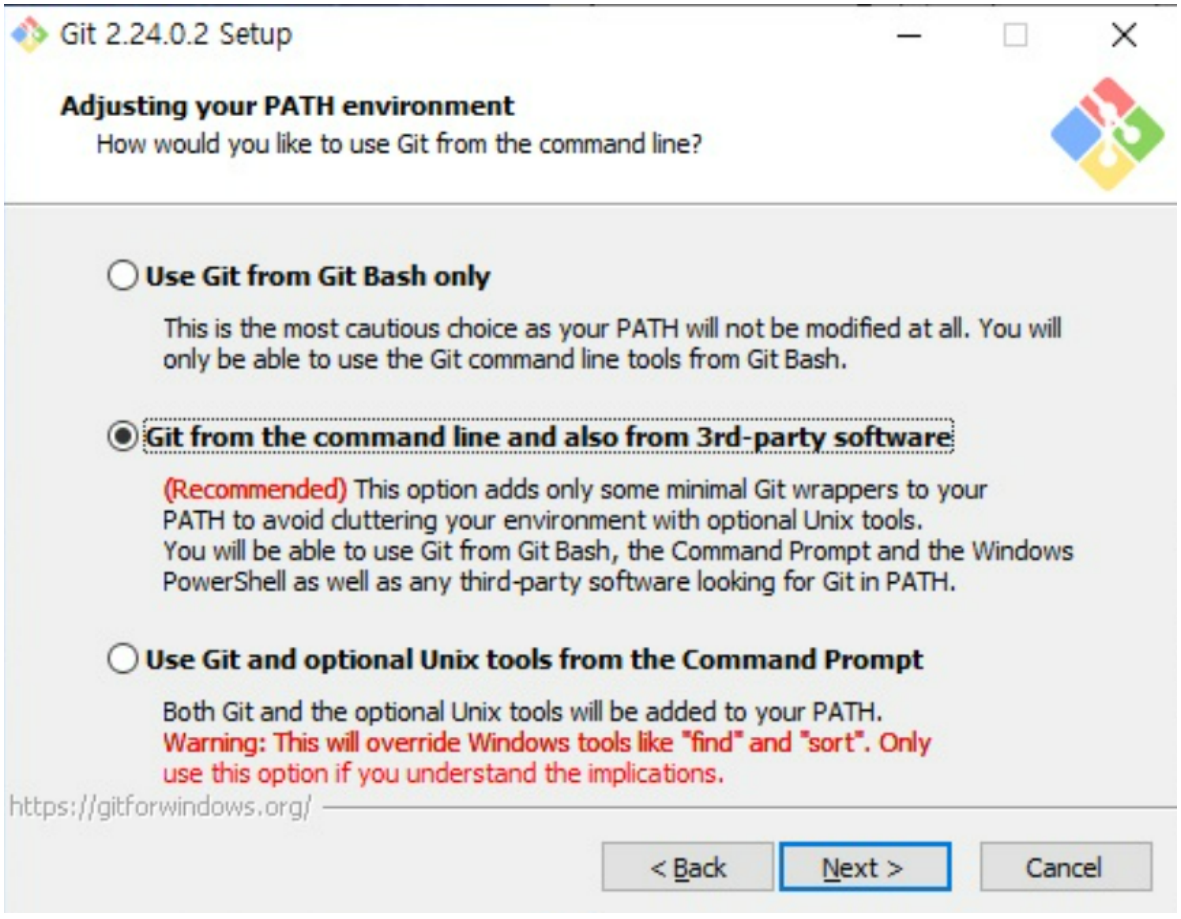
공식 사이트(https://git-scm.com/download/win)에서 Windows 용 64-bit Standalone Installer 를 내려받아 기본값으로 설치합니다.



[그림 24] Git for Windows 다운로드 페이지

PATH 설정 화면에서 "Git from the command line and also from 3rd-party software"(기본값)를 선택합니다.

줄바꿈 설정은 "Checkout Windows-style, commit Unix-style line endings"(기본값)을 유지합니다.



[그림 25] Git 설치 및 PATH 설정

2) JDK 17 (Eclipse Temurin)

○ **명령어로 설치**

```
winget install -e --id EclipseAdoptium.Temurin.17.JDK --accept-package-agreements --accept-source-agreements
```

설치가 완료되면, 아래 버전 확인으로 설치 여부를 확인합니다.

```
java -version
```

JAVA_HOME 을 사용자 환경변수로 등록합니다. (폴더명은 위 dir 결과로 교체)

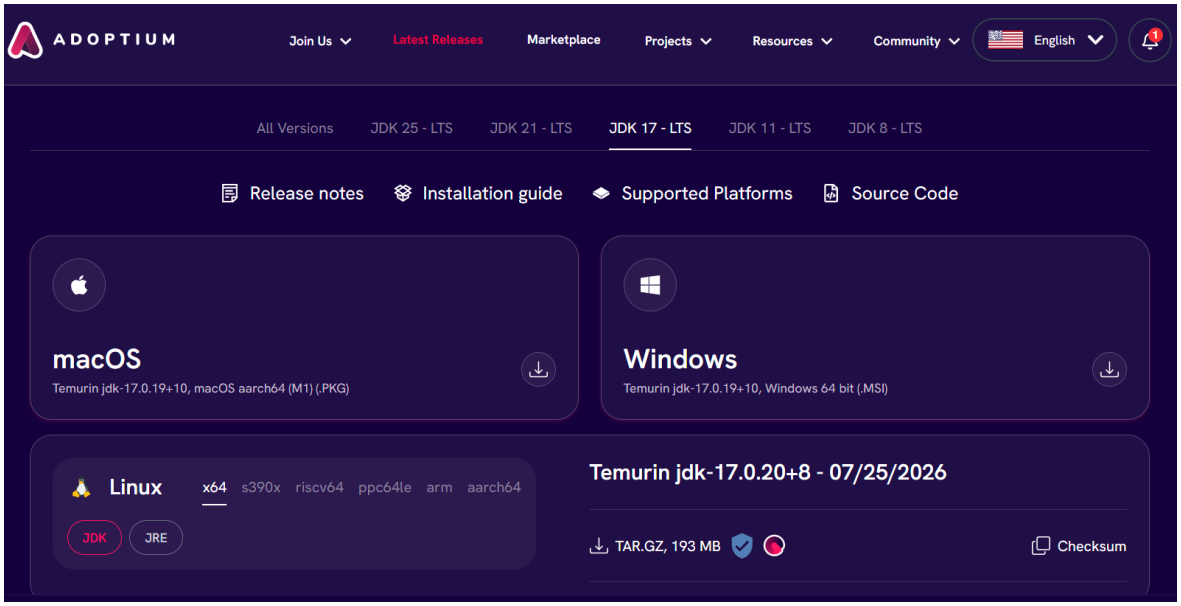
후미의 jdk 파일명은 다를 수 있으므로, 확인 후 올바른 파일명을 입력합니다.

```
setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-17.0.11.9-hotspot"
```

> ※ setx 는 새로 여는 명령창부터 적용됩니다. 설정 후 명령창을 닫았다가 다시 열어 java -version 이 17 로 나오는지 확인하십시오.
>
> ※ 다른 Java(예: 11)가 PATH 앞쪽에 있어 계속 충돌한다면, PATH 를 setx 로 덮어쓰기보다(기존 값이 잘리는 사고가 발생할 수 있음) 아래와 같이 빌드 시에만 JDK 를 지정하는 편이 안전합니다.

○ **다운로드 페이지에서 설치**

Adoptium 사이트(https://adoptium.net/temurin/releases/?version=17)에서 Version 17 / Windows / x64 / JDK 를 선택해 `.msi` 를 내려받습니다.

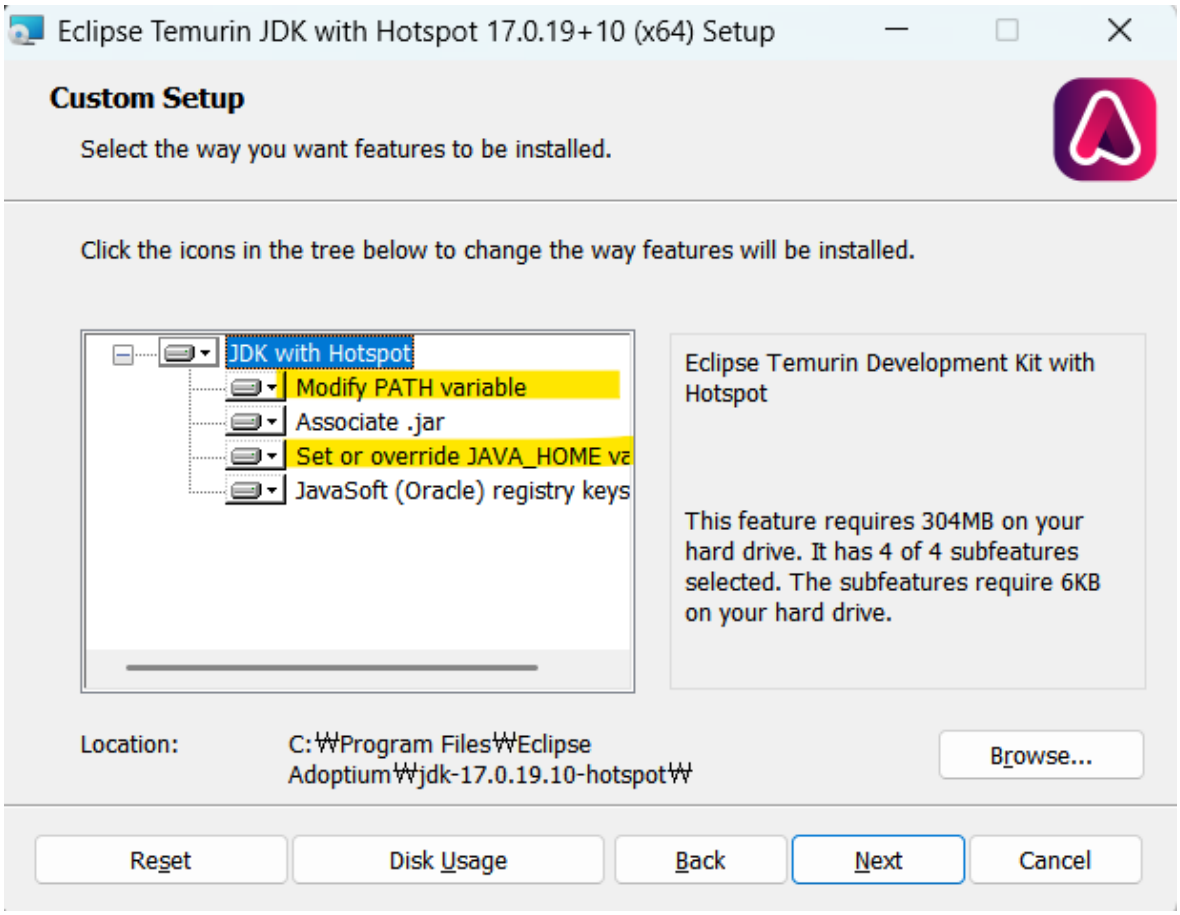


[그림 26] Adoptium Temurin 17(JDK) 다운로드

설치 옵션(Custom Setup) 화면에서 아래 두 항목을 반드시 활성화합니다.

Set JAVA_HOME variable — `JAVA_HOME` 환경변수 자동 설정

Modify PATH variable — `java` 명령을 명령창에서 사용하도록 등록



[그림 27] Temurin 설치 — JAVA_HOME / Modify PATH 활성화

> ※ PC 에 다른 Java(예: JDK 11)가 설치되어 있으면 PATH 순서에 따라 버전이 충돌할 수 있습니다. `JAVA_HOME` 과 PATH 가 17 을 우선하도록 설정합니다.

3) Node.js

- **명령어로 설치**

```
winget install -e --id OpenJS.NodeJS.LTS --accept-package-agreements --accept-source-agreements


node --version
npm --version
```

- **다운로드 페이지에서 설치**

공식 사이트(https://nodejs.org)에서 LTS 버전(22.x) Windows Installer(`.msi`, 64-bit)를 내려받아 기본 옵션으로 설치합니다. npm 이 함께 설치됩니다.



[그림 28] Node.js LTS 다운로드

> ※ "Automatically install the necessary tools(Tools for Native Modules)" 항목은 본 프로젝트에서 필요 없습니다.
>
> ※ 여러 Node 버전을 오가야 하는 경우 nvm-windows 사용을 고려합니다.

4) Docker Desktop (+ WSL2)

○ 명령어로 설치

Docker Desktop 은 WSL2 기반으로 실행되므로, 설치 전에 Windows 의 WSL 기능과 가상 머신 플랫폼을 활성화하고 Hypervisor 가 부팅 시 자동 실행되도록 설정합니다.

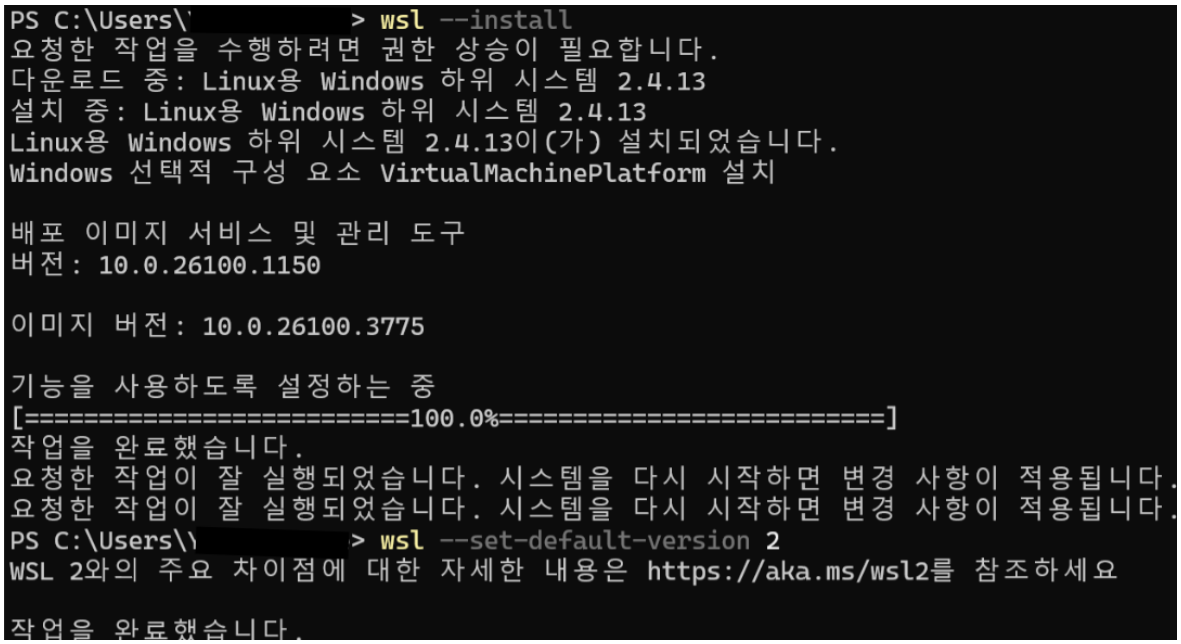
관리자 권한 명령 프롬프트(CMD)에서 아래 명령을 실행합니다.

```
dism /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart
dism /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
bcdedit /set hypervisorlaunchtype auto
shutdown /r /t 0
```

재부팅 후 관리자 권한 명령 프롬프트(CMD)를 다시 열고 WSL2 를 설치·갱신합니다.

```
wsl --install
wsl --update
wsl --set-default-version 2
```

※ --no-launch 옵션은 Ubuntu 최초 실행 시 사용자 계정 및 metrics 입력 화면



[그림 29] wsl --install --no-l

WSL2 준비에 이어 Docker Desktop 도 명령으로 설치합니다. 관리자 권한 명령창에서 실행하고, 완료 후 재부팅 또는 재로그인을 요구하

```
winget install -e --id Docker.DockerDesktop --accept-package-agreements --accept-source-agreements
```

설치 후 Docker Deskt

```
start "" "C:\Program Files\Docker\Docker\Docker Desktop.exe"


docker --version
docker compose version
docker info
docker run --rm hello-world
```

[Settings] → [Resources]에서 조정할 수 있는 WSL2 메모리 할당은, 본 문서에서는 명령 프롬프트로 재현 가능한 .wslconfig 파일 방식으로 설정합니다.

```
> "%USERPROFILE%\.wslconfig" echo [wsl2]
>> "%USERPROFILE%\.wslconfig" echo memory=8GB
>> "%USERPROFILE%\.wslconfig" echo processors=4

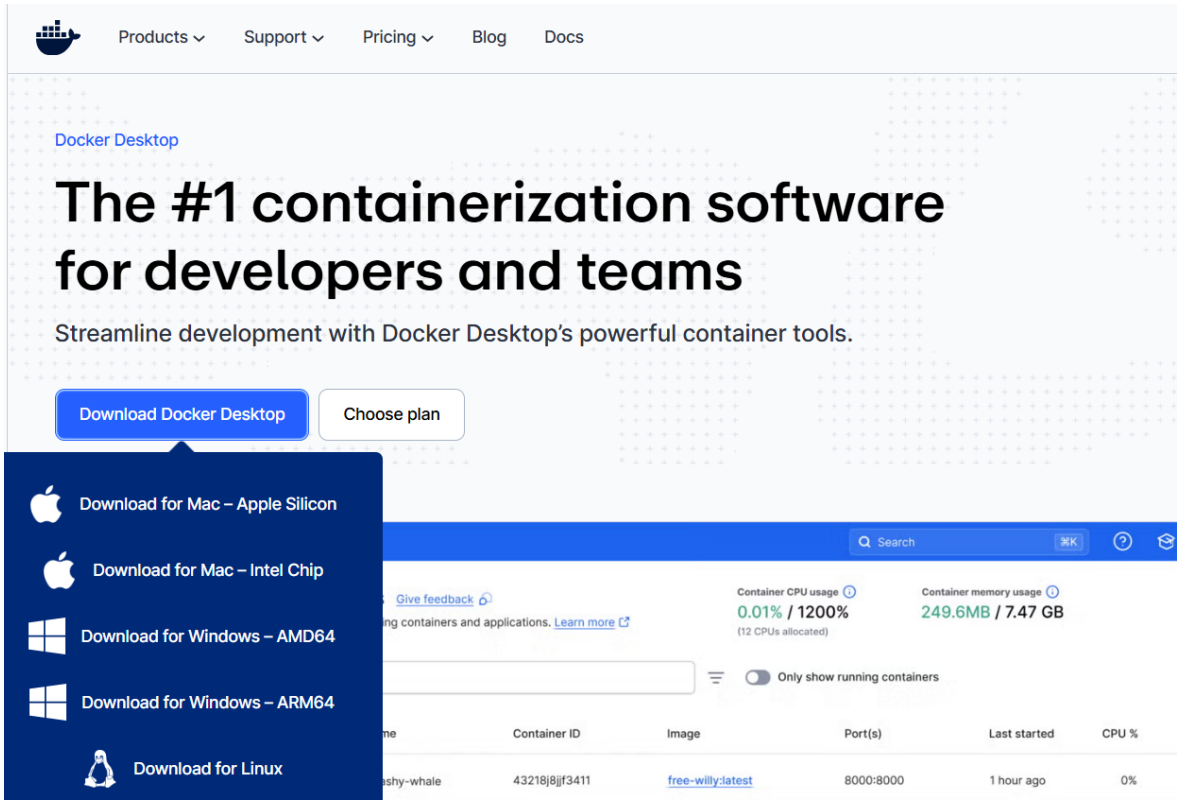

type "%USERPROFILE%\.wslconfig"
wsl --shutdown
```

※ 위 명령은 사용자 프로필 폴더에 .wslconfig 파일을 생성하고, WSL2 에 memory=8GB 및 processors=4 를 지정합니다. Docker Desktop 이 WSL2 기반으로 컨테이너를 실행하므로 이후 4 장 인프라 컨테이너 구동 시 메모리 부족으로 컨테이너가 재시작되는 위험을 줄입니다.

※ 설정은 wsl --shutdown 후 Docker Desktop 을 다시 실행해야 반영됩니다. 단, 실제 PC 메모리가 부족하거나 다른 프로그램이 메모리를 많이 사용하면 8GB 지정만으로 모든 메모리 부족을 보장하지는 않으므로, 구동 후 docker stats 로 사용량을 확인하십시오.

○ **다운로드 페이지에서 설치**

공식 사이트(https://www.docker.com/products/docker-desktop/)에서 "Download for Windows (AMD64)" 설치 파일을 내려받습니다.



[그림 30] Docker Desktop 다운로드 페이지

설치 마법사에서 "Use WSL 2 instead of Hyper-V (recommended)" 옵션을 유지한 채 설치하고, 완료 후 재부팅 또는 재로그인합니다.



[그림 31] Docker Desktop 설치 — WSL2 사용 옵션

※ [Settings] → [Resources] 에서 **메모리를 8GB 이상 할당**합니다.

◆ ※ WSL2 는 CPU 가상화 기능이 켜져 있어야 설치됩니다. wsl --install --no-launch 이 실패하면 PC 를 재시작해 BIOS/UEFI 에서 Intel VT-x 또는 AMD-V(SVM) 를 활성화한 뒤 다시 시도하십시오. 현재 상태는 작업 관리자 → [성능] → [CPU] 의「가상화」항목에서

◆ ※ Docker Desktop 을 처음 실행하면 서비스 약관 동의 화면이 표시됩니다. 동의해야 엔진이 기동되며 계정 로그인은 하지 않아도 됩니다. 엔진이 준비되기 전에는 docker 명령이 Cannot connect to the Docker daemon 오류로 실패하므로, docker info 가 정상 출력되는지 먼저 확인하십시오.

5) IntelliJ IDEA

○ **명령어로 설치**

```
winget install -e --id JetBrains.IntelliJIDEA.Community --accept-package-agreements --accept-source-agreements
```

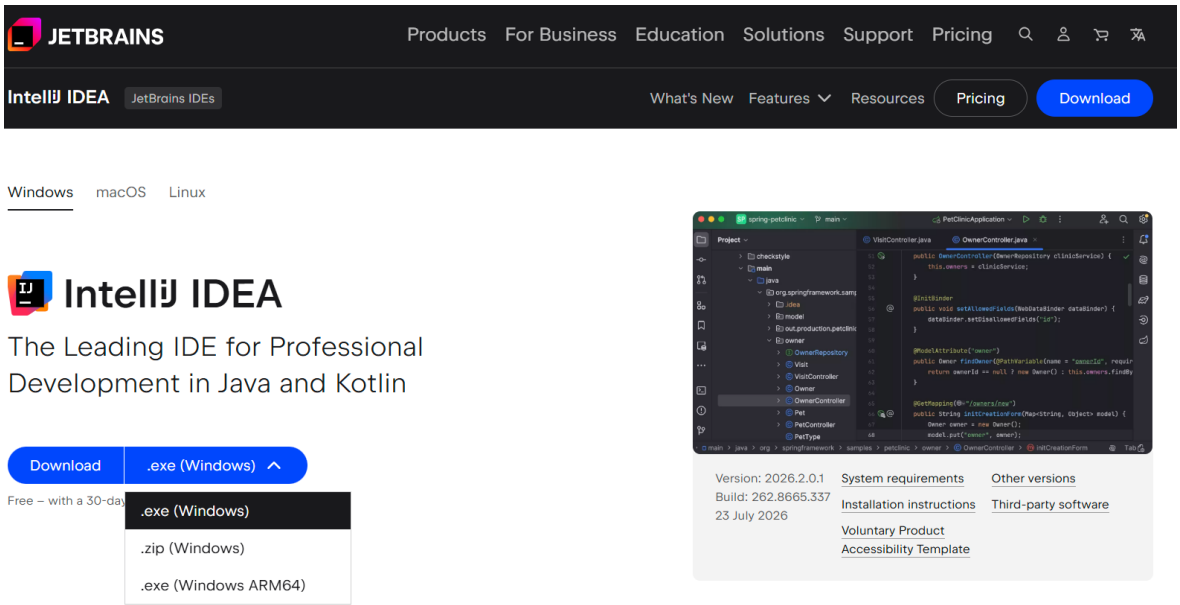
※ 백엔드 실행·빌드는 IntelliJ 없이 gradlew.bat 만으로도 수행할 수 있습니다. 소스를 직접 열어 수정할 때만 설치하면 됩니다. (4 장「나」참고)

본 해설서의 설명과정에 사용된 IDE 는 IntelliJ 를 기반으로 서술되었기 때문에,

IDE 는 IntelliJ 설치를 권장합니다.

○ **다운로드 페이지에서 설치**

JetBrains 사이트(https://www.jetbrains.com/idea/download/)에서 Windows(.exe)를 내려받아 기본 옵션으로 설치합니다.



[그림 32] IntelliJ IDEA 다운로드

○ **최초 실행 설정(FeDit.git clone 이후)**

[File] → [Open] 에서 BackEnd₩src₩searchRule 폴더(build.gradle 이 있는 폴더)를 선택해 엽니다.

[File] → [Project Structure] → [Project] 의 SDK 를 JDK 17 로 지정합니다.

[File] → [Settings] → [Build, Execution, Deployment] → [Build Tools] → [Gradle] 의 Gradle JVM 도 JDK 17 로 지정합니다.

[File] → [Settings] → [Build, Execution, Deployment] → [Compiler] → [Annotation Processors] 에서 Enable annotation processing 을 체크합니다.

※ 백엔드는 Lombok 을 사용합니다. gradlew.bat bootRun / bootJar 등으로 명령 프롬프트를 통한 실행 시에는 build.gradle 의 annotationProcessor 설정으로 Lombok 이 자동 처리되므로 별도 설치가 필요 없으나,

IntelliJ 에서 getter/setter/log 관련 오류가 표시되면 [Settings] → [Build, Execution, Deployment] → [Compiler] → [Annotation Processors]에서 Enable annotation processing 을 체크합니다.

> ※ Project SDK 와 Gradle JVM 은 별개 설정입니다. Project SDK 만 17 로 맞추고 Gradle JVM 이 다른 JDK 로 남아 있으면 빌드가 실패합니다.
>
> ※ Annotation processing 을 켜지 않으면 Lombok 이 생성하는 코드(getter · setter 등)를 찾지 못해 컴파일 오류가 발생합니다.

6) Visual Studio Code

- **명령어로 설치**

```
winget install -e --id Microsoft.VisualStudioCode --accept-package-agreements --accept-source-agreements
```

- **다운로드 페이지에서 설치**

공식 사이트(https://code.visualstudio.com)에서 Windows 용 User Installer(64-bit)를 내려받습니다. "Add to PATH"와 "Open with Code" 옵션을 권장합니다.



[그림 33] Visual Studio Code 다운로드

권장 확장 프로그램	용도
ESLint	`eslint.config.js` 기반 정적 검사
Prettier - Code formatter	`.prettierrc` 기반 코드 자동 포맷
Tailwind CSS IntelliSense	Tailwind 클래스 자동완성

7) DBeaver

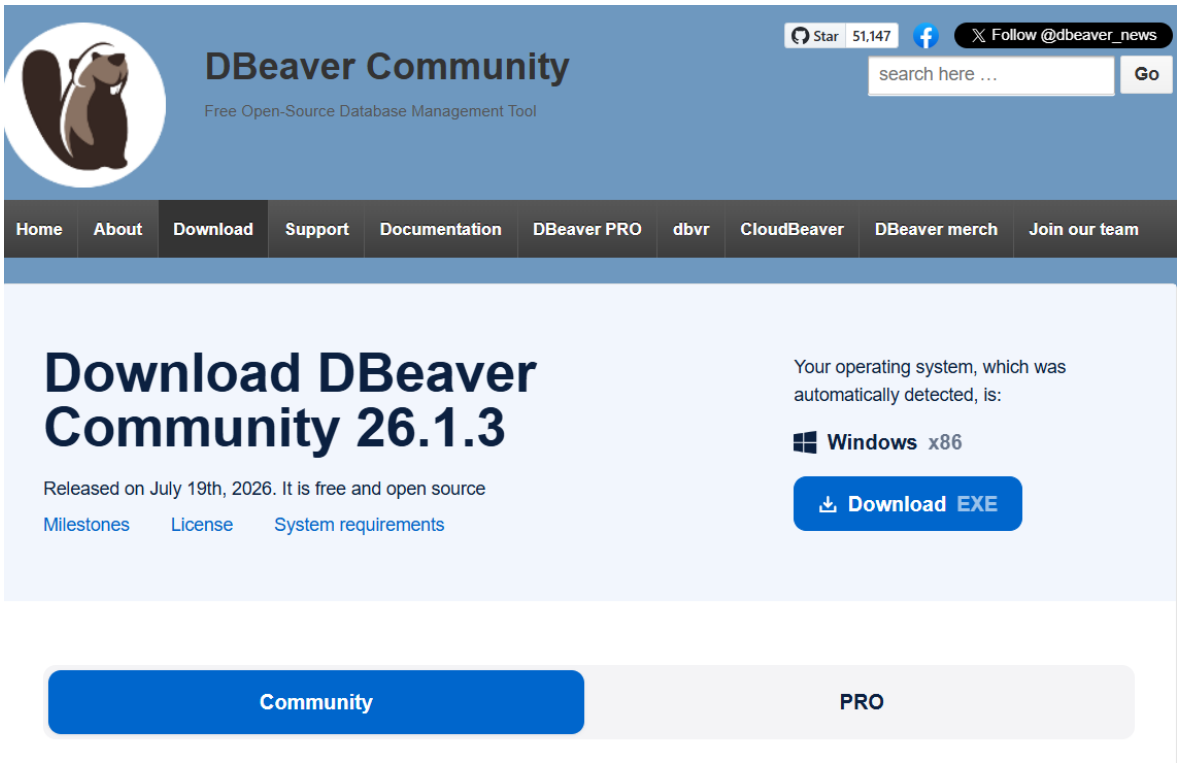
○ 명령어로 설치

```
winget install -e --id DBeaver.DBeaver.Community --accept-package-agreements --accept-source-agreements
```

> ※ DBeaver 를 설치하지 않고 명령만으로 DB 를 확인하려면, 기동 중인 컨테이너 내부의 클라이언트를 이용할 수 있습니다.

○ 다운로드 페이지에서 설치

공식 사이트(https://dbeaver.io/download/)에서 Community Edition Windows Installer 를 내려받습니다.



[그림 34] DBeaver Community 다운로드

대상	호스트	포트	DB	계정
Stellio PostgreSQL	localhost	5432	`stellio_search`	`stellio` / `stellio_password`
ontology-converter PostgreSQL	localhost	6543	`ontology_converter`	`ontology_converter` / `ontology1234`

> ※ Neo4j 는 컨테이너에 Neo4j Browser 가 내장되어 있어 `http://localhost:7474` 로 접속하므로 별도 클라이언트가 필요 없습니다.

8) 7-Zip

- **명령어로 설치**

```
winget install -e --id 7zip.7zip --accept-package-agreements --accept-source-agreements
```

- **다운로드 페이지에서 설치**

공식 사이트(https://www.7-zip.org)에서 Windows 64-bit 설치 파일을 내려받아 기본값으로 설치합니다.

> ※ Windows 기본 압축 도구는 경로가 긴 파일에서 오류가 발생합니다. 저장소 압축본이나 대용량 산출물을 다룰 때 사용합니다.

9) SSH · SFTP 접속 도구

서버 접속(SSH)은 PuTTY, 운용 폴더 업로드(SFTP)는 WinSCP 를 기준으로 합니다. 두 도구 모두 같은 서버 IP, SSH 포트, 계정 정보를 사용하며, 아래 명령은 관리자 권한 명령 프롬프트에서 실행합니다.

- **명령어로 설치**

```
winget install -e --id PuTTY.PuTTY --accept-package-agreements --accept-source-agreements
winget install -e --id WinSCP.WinSCP --accept-package-agreements --accept-source-agreements
```

○ 설치 후 확인

설치 후 시작 메뉴에서 PuTTY 와 WinSCP 가 검색되는지 확인합니다.

> ※ WinSCP 패키지 ID 가 조회되지 않으면 winget search WinSCP 로 현재 표시되는 ID 를 확인한 뒤 --id 값을 바꾸어 설치하십시오.
>
> ※ Windows 10 1809 이상에는 ssh 명령이 기본 포함되어 있지만, 본 문서의 서버 접속 도구 설치는 PuTTY 를 기준으로 설명합니다. (확인: where ssh → C:₩Windows₩System32₩OpenSSH₩ssh.exe)
>
> ※ 접속 절차는 6 장「나」1), 운용 폴더 업로드는 6 장「다」를 참고하십시오.

이 부분까지 도구 설치가 완료되었다면, 13 페이지의 2 장 '나' - '설치 후 버전 확인'으로 돌아가서 정상 설치여부를 체크합니다.

마. [대안] Docker 미사용 시 인프라 직접 설치

> **이 절은 대부분의 경우 건너뜁니다.**

기본(권장) 운용은 Docker Compose 이며, Kafka·Neo4j·PostgreSQL·Stellio 는 `docker compose up` 시 컨테이너로 자동 실행됩니다. 본 절은 Docker 를 사용할 수 없어(예: 폐쇄망) 이 구성요소들을 호스트에 직접 설치·구동해야 할 때만 참조합니다.

기동 순서는 **PostgreSQL → Kafka → Neo4j → Stellio** 를 권장하며, 계정·포트·DB 명은 운용 폴더의 `.env` 및 `docker-compose.yml` 기준값과 동일하게 맞춥니다.

구성요소	버전	설치 방법(요약)	핵심 설정
PostgreSQL (Stellio 용)	16	EDB 설치 + PostGIS·TimescaleDB 확장	DB 2 개(`stellio_search`/`stellio_subscription`), 포트 5432
PostgreSQL (변환기용)	17	EDB 설치(별도 인스턴스)	DB `ontology_converter`, 포트 6543
Neo4j	5.22.0	zip 압축 해제	APOC·n10s 플러그인, 초기 PW `test1234`, 7474/7687
Apache Kafka	3.8.0	tgz 압축 해제	KRaft 포맷 후 기동, 9092
Stellio	`STELLIO_DOCKER_TAG` 기준	소스 빌드 후 JAR 실행	PG·Kafka 선행, 8080/8083/8084

> ※ 폐쇄망에서는 2 장「가」에서 언급한 **Spring Boot 스냅샷 의존성 문제**가 함께 발생합니다. 백엔드 빌드에 필요한 라이브러리를 사전에 확보해 두어야 합니다.
>
> ※ Stellio 는 설치 프로그램이 없어 소스(https://github.com/stellio-hub/stellio-context-broker)를 빌드해 JAR 로 구동합니다.

바. 자주 사용하는 명령어

○ 작업 PC(빌드 단계) 명령어

위 표는 운용 서버(6 장) 기준입니다. 작업 PC 에서 소스를 내려받아 빌드하기까지 자주 쓰는 명령은 아래와 같습니다.

```
# == 공통 / 환경 확인 ========================
git --version & java -version & node --version & docker --version
echo %JAVA_HOME%
where java


# == 소스 (3 장) ================================
git pull origin main                      # 최신 소스 반영
git status                                # 로컬 변경 확인
git log -1 --oneline                      # 현재 커밋 확인


# == 인프라 (4 장 가) ===========================
docker compose up -d                      # 인프라 기동
docker compose ps                         # 상태 확인
docker compose logs --tail 100 {서비스명}   # 로그 확인
docker compose down                       # 중지 + 삭제


# == 백엔드 (4 장) ==============================
gradlew.bat clean bootJar                  # 빌드
gradlew.bat bootRun                       # 로컬 실행
gradlew.bat --stop                        # Gradle 데몬 종료
java -jar build\libs\searchRule-0.0.1-SNAPSHOT.jar


# == 프론트엔드 (5 장) ==========================
npm install                               # 패키지 설치
npm run dev                               # 로컬 구동
npm run build                             # 배포용 빌드
npm run preview                           # 빌드 결과 확인


# == 포트 / 프로세스 ============================
netstat -ano | findstr /C:":8888 "         # 포트 점유 확인 (맨 오른쪽이 PID)
taskkill /PID {PID} /F                    # 해당 프로세스 강제 종료
tasklist | findstr java                   # 실행 중인 java 프로세스 확인


# == 산출물 반영 ================================
copy /Y BackEnd\src\searchRule\build\libs\searchRule-0.0.1-SNAPSHOT.jar
initResource\OntologyConverter\
xcopy FrontEnd\src\ontology-converter-frontend\dist
initResource\OntologyConverter\frontend\dist\ /E /I /Y
```

○ 운용 서버 명령어

구분	명령어	설명
상태 확인	`docker compose ps`	컨테이너 목록·상태 확인
로그 확인	`docker compose logs -f {서비스명}`	실시간 로그 확인 (종료: `Ctrl + C`)
로그 확인	`docker compose logs --tail 100 {서비스명}`	최근 100 줄만 확인
재기동	`docker compose restart {서비스명}`	특정 서비스만 재기동

구분	명령어	설명
내부 접속	`docker compose exec {서비스명} bash`	컨테이너 내부 접속 (종료: `exit`)
이미지 확인	`docker images`	생성된 이미지 목록 확인
볼륨 확인	`docker volume ls`	생성된 볼륨 목록 확인
자원 사용량	`docker stats`	컨테이너별 CPU / 메모리 사용량
포트 확인	`sudo lsof -i :{포트}`	해당 포트를 사용 중인 프로그램 확인

사. 다운로드 링크 총괄

○ winget 패키지 ID 로 설치

위 표의 다운로드 페이지를 방문하지 않고, 아래 ID 로 바로 설치할 수 있습니다. 관리자 권한 명령창에서 실행합니다.

```
winget install -e --id Git.Git --accept-package-agreements --accept-source-agreements
winget install -e --id EclipseAdoptium.Temurin.17.JDK --accept-package-agreements --accept-
source-agreements
winget install -e --id OpenJS.NodeJS.LTS --accept-package-agreements --accept-source-
agreements
winget install -e --id Docker.DockerDesktop --accept-package-agreements --accept-source-
agreements
winget install -e --id JetBrains.IntelliJIDEA.Community --accept-package-agreements --
accept-source-agreements
winget install -e --id Microsoft.VisualStudioCode --accept-package-agreements --accept-
source-agreements
winget install -e --id DBeaver.DBeaver.Community --accept-package-agreements --accept-
source-agreements
winget install -e --id 7zip.7zip --accept-package-agreements --accept-source-agreements
winget install -e --id PuTTY.PuTTY --accept-package-agreements --accept-source-agreements
winget install -e
```

ID 가 바뀌었거나 확실하지 않은 경우 검색으로 확인할 수 있습니다.

```
winget search temurin
winget search nodejs
winget list                                  # 설치된 목록 확인
winget upgrade --all                          # 설치된 도구 일괄 업데이트
```

> ※ Kafka·Neo4j·PostgreSQL·Stellio 는 docker compose 로 컨테이너 기동되므로 작업 PC 에 별도 설치가 필요 없습니다. (부록 「마」의 직접 설치는 Docker 를 쓸 수 없는 경우에만 해당)

○ 공식 다운로드 링크

도구 / 구성요소	공식 다운로드 URL
Git	https://git-scm.com/download/win
JDK 17 (Temurin)	https://adoptium.net/temurin/releases/?version=17
Node.js	https://nodejs.org
Docker Desktop	https://www.docker.com/products/docker-desktop/
IntelliJ IDEA	https://www.jetbrains.com/idea/download/
Visual Studio Code	https://code.visualstudio.com
DBeaver	https://dbeaver.io/download/
PostgreSQL (Windows)	https://www.postgresql.org/download/windows/
TimescaleDB	https://docs.timescale.com/self-hosted/latest/install/installation-windows/

도구 / 구성요소	공식 다운로드 URL
(Windows)	
Neo4j	https://neo4j.com/deployment-center/
APOC 플러그인	https://github.com/neo4j/apoc/releases
neosemantics(n10s)	https://github.com/neo4j-labs/neosemantics/releases
Apache Kafka 3.8.0	https://archive.apache.org/dist/kafka/3.8.0/
Stellio Context Broker	https://github.com/stellio-hub/stellio-context-broker

아. 설정 항목 일람

운용 환경 변경 시 아래 항목을 일괄 확인하십시오.

구분	설정 위치	비고
웹 UI 접속 포트	`docker-compose.yml` (`8181:80`)	이용자 PC 에서 접근 가능해야 함
Neo4j 포트	`docker-compose.yml` (`7474`, `7687`)	이용자 PC 에서 접근 가능해야 함 (그래프 화면)
fedit API 주소	`application-docker.properties` (`fedit.api.base-url`)	초기 적재 원본
CORS 허용 출처	`application-docker.properties` (`cors.allowed-origins`)	현재 `33000` 포트
Neo4j 계정	`.env`(`NEO4J_AUTH`) · 백엔드 · 프론트 3 곳	세 곳이 일치해야 함
변환기 DB 계정	`.env`(`OC_DB_PASS`) · 백엔드	두 곳이 일치해야 함
Stellio 이미지 태그	`.env`(`STELLIO_DOCKER_TAG`)	현재 `latest-dev`
TTL 출력 경로	`docker-compose.yml` (`./output`)	폴더 사전 생성 필요

> ※ 위 항목이 서로 다른 값을 가리키면 화면은 표시되나 데이터가 조회되지 않습니다. 운용 서버를 변경할 때는 **표의 모든 항목을 함께** 확인하십시오.

자. 인터넷이 차단된 환경에서의 설치 (폐쇄망)

운용 서버가 인터넷에 접속할 수 없는 경우, 필요한 패키지와 컨테이너 이미지를 **인터넷이 가능한 장비에서 미리 내려받아 반입**해야 합니다.

○ 1) Docker 설치 패키지 반입

인터넷이 가능한 **동일 버전의 Rocky 9 장비**에서 수행합니다.

```
sudo dnf install -y dnf-plugins-core
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo


mkdir -p ~/docker-rpms && cd ~/docker-rpms
sudo dnf download --resolve --alldeps \
  docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin \
  policycoreutils-python-utils chrony


tar czf docker-rpms.tar.gz *.rpm
```

운용 서버로 전송한 뒤 설치합니다.

```
tar xzf docker-rpms.tar.gz
sudo dnf install -y ./*.rpm
sudo systemctl enable --now docker
```

○ 2) 컨테이너 이미지 반입

이 시스템이 사용하는 이미지는 아래 9 종입니다.

인터넷이 가능한 장비에서 내려받아 하나의 파일로 묶습니다.

이미지	사용처
stellio/stellio-api-gateway:latest-dev	Stellio 진입점
stellio/stellio-search-service:latest-dev	Stellio 엔티티 조회
stellio/stellio-subscription-service:latest-dev	Stellio 구독 관리
stellio/stellio-timescale-postgis:16-2.20.2-3.5	Stellio PostgreSQL
apache/kafka:3.8.0	메시지 브로커
neo4j:5.22.0	그래프 DB
postgres:17	변환기 PostgreSQL
eclipse-temurin:17-jre	백엔드 이미지의 기반 (Dockerfile)
nginx:alpine	프론트 이미지의 기반 (frontend/Dockerfile)

```
docker pull stellio/stellio-api-gateway:latest-dev
docker pull stellio/stellio-search-service:latest-dev
docker pull stellio/stellio-subscription-service:latest-dev
docker pull stellio/stellio-timescale-postgis:16-2.20.2-3.5
docker pull apache/kafka:3.8.0
docker pull neo4j:5.22.0
docker pull postgres:17
docker pull eclipse-temurin:17-jre
docker pull nginx:alpine


docker save -o ontology-images.tar \
  stellio/stellio-api-gateway:latest-dev \
  stellio/stellio-search-service:latest-dev \
  stellio/stellio-subscription-service:latest-dev \
  stellio/stellio-timescale-postgis:16-2.20.2-3.5 \
  apache/kafka:3.8.0 neo4j:5.22.0 postgres:17 \
  eclipse-temurin:17-jre nginx:alpine
```

운용 서버로 전송합니다. **(작업 PC 에서 입력)**

```
scp -P {sshPort} ontology-images.tar {계정}@{serverIp}:~/
```

서버에서 적재합니다.

```
docker load -i ~/ontology-images.tar
docker images
```

> ※ 파일 크기가 수 GB 에 이르므로 전송 시간과 디스크 여유를 미리 확인하십시오.
>
> ※ eclipse-temurin:17-jre 와 nginx:alpine 이 없으면 6 장 「마」의 docker compose up -d --build 에서 백엔드·프론트 이미지 생성이 실패합니다.
>
> ※ .env 의 STELLIO_DOCKER_TAG 가 latest-dev 이므로, 반입한 이미지와 태그가 일치해야 합니다. 폐쇄망에서는 태그를 특정 릴리스로 고정하는 편이 안전합니다.

차. 프록시 환경에서의 Docker 설정

사내 프록시를 거쳐야 외부에 접속할 수 있는 환경에서는 Docker 데몬에 프록시를 별도로 지정해야 합니다. 사용자 계정의 환경변수(export HTTP_PROXY=...)만 설정하면 docker pull 은 여전히 실패합니다.

```
sudo mkdir -p /etc/systemd/system/docker.service.d


sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf > /dev/null <<'EOF'
[Service]
Environment="HTTP_PROXY=http://{프록시주소}:{프록시포트}"
Environment="HTTPS_PROXY=http://{프록시주소}:{프록시포트}"
Environment="NO_PROXY=localhost,127.0.0.1,::1"
EOF


sudo systemctl daemon-reload
sudo systemctl restart docker


sudo systemctl show docker --property=Environment
docker pull busybox                    # 정상적으로 내려받아지면 설정 완료
```

> ※ {프록시주소} · {프록시포트} 는 네트워크 담당자에게 확인합니다.
>
> ※ 프록시에 인증이 필요한 경우 http://{계정}:{비밀번호}@{프록시주소}:{프록시포트} 형식으로 입력합니다. 이 파일에는 비밀번호가 평문으로 저장되므로 sudo chmod 600 /etc/systemd/system/docker.service.d/http-proxy.conf 로 권한을 제한하십시오.

카. 사내망(프록시) 환경에서의 작업 PC 설정

사내 프록시를 거쳐야 외부에 접속할 수 있는 PC 에서는 도구마다 프록시를 각각 지정해야 합니다. 지정하지 않으면 소스 내려받기와 의존성 다운로드가 모두 실패합니다. {프록시주소} · {프록시포트} 는 네트워크 담당자에게 확인합니다.

○ Git

```
git config --global http.proxy http://{프록시주소}:{프록시포트}
git config --global https.proxy http://{프록시주소}:{프록시포트}
git config --global --get-regexp proxy      # 설정 확인
```

○ npm

```
npm config set proxy http://{프록시주소}:{프록시포트}
npm config set https-proxy http://{프록시주소}:{프록시포트}
npm config list                            # 설정 확인
```

○ Gradle

```
if not exist "%USERPROFILE%\.gradle" mkdir "%USERPROFILE%\.gradle"


> "%USERPROFILE%\.gradle\gradle.properties" echo systemProp.http.proxyHost={프록시주소}
>> "%USERPROFILE%\.gradle\gradle.properties" echo systemProp.http.proxyPort={프록시포트}
>> "%USERPROFILE%\.gradle\gradle.properties" echo systemProp.https.proxyHost={프록시주소}
>> "%USERPROFILE%\.gradle\gradle.properties" echo systemProp.https.proxyPort={프록시포트}


type "%USERPROFILE%\.gradle\gradle.properties"
```

○ **Docker Desktop**

[Settings] → [Resources] → [Proxies] 에서 프록시 주소를 지정합니다. 지정하지 않으면 컨테이너 이미지 내려받기가 실패합니다.

> ※ 인증이 필요한 프록시는 http://{계정}:{비밀번호}@{프록시주소}:{프록시포트} 형식으로 입력합니다. 설정 파일에 비밀번호가 평문으로 저장되므로 파일 접근 권한에 주의하십시오.
>
> ※ 사내에서 SSL 검사 장비를 사용하는 경우 self signed certificate in certificate chain 오류가 발생합니다. 사내 CA 인증서를 받아 등록해야 하며, 검증을 끄는 설정(strict-ssl false 등)은 보안상 권장하지 않습니다.

프록시를 사용하지 않는 환경으로 옮길 때는 아래로 해제합니다.

```
git config --global --unset http.proxy
git config --global --unset https.proxy
npm config delete proxy
npm config delete https-proxy
```

> ※ 운용 서버의 Docker 프록시 설정은「차. 프록시 환경에서의 Docker 설정」을 참고하십시오.